Data Structures

22. B-Trees

Handling Very Large Trees

- So far we have assumed that an entire data structure can be stored in main memory
- What if we have more data than can fit in main memory?
  - Data structure must reside on disk
- Time complexity analysis assumes that all operations take roughly same time
  - This assumption is not true when disk access (I/O) is involved
  - Time complexity analysis (Big-Oh notation) fails!

Outline

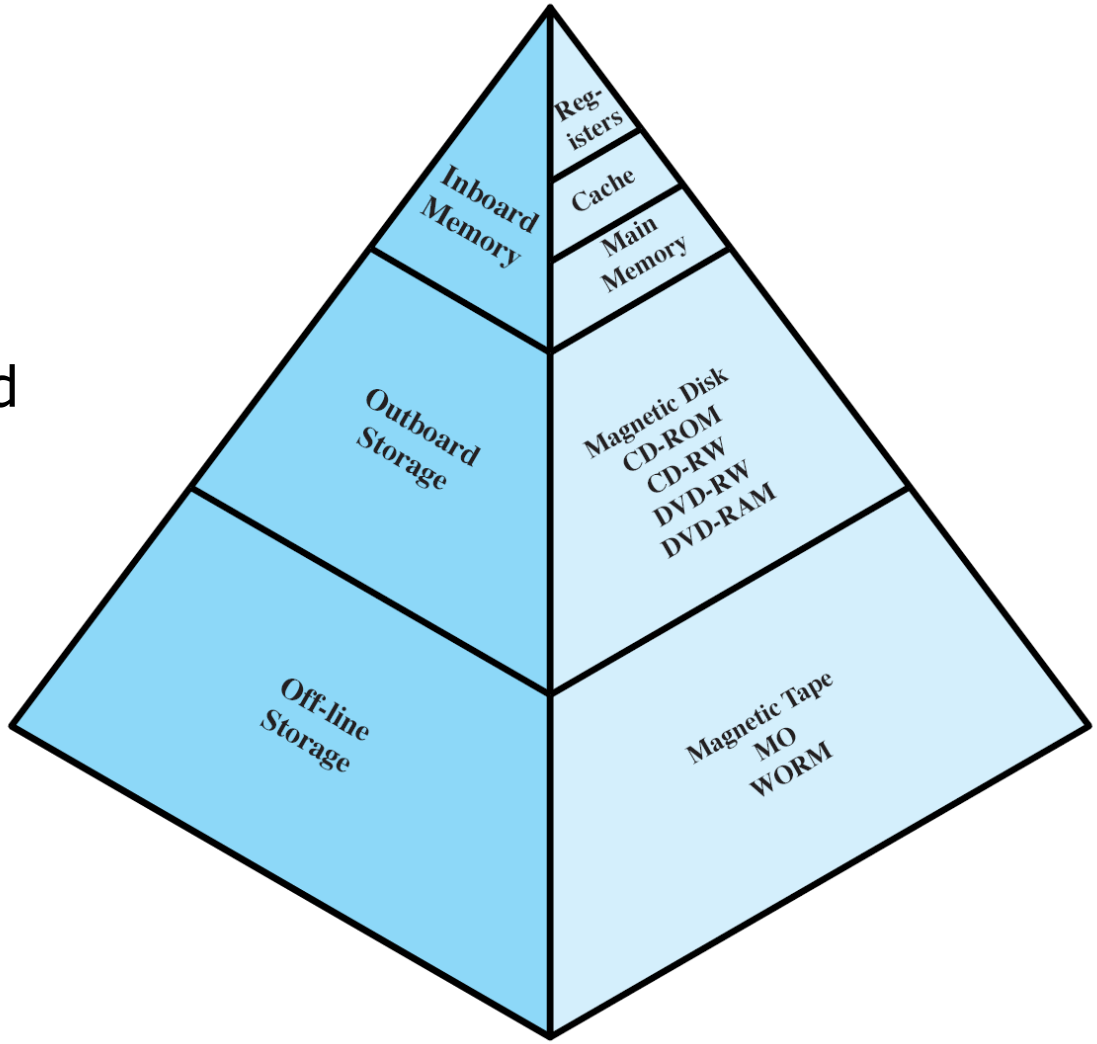
- Motivation
  - Hard drives accesses are slow
  - Reduces the number of disk accesses

- B trees:
  - Description
  - Insertion and deletions

Memory Hierarchy

- Design criteria
  - Capacity
  - Access time
  - Cost to buy / produce

- Following relationships hold
  - Faster access, higher cost per bit
  - Greater capacity, smaller cost per bit
  - Greater capacity, slower access time



Memory Hierarchy

- Main memory is limited: MiB-GiB
- Caches even more so: KiB-MiB
- Hard drives, however, are rich: GiB-TiB

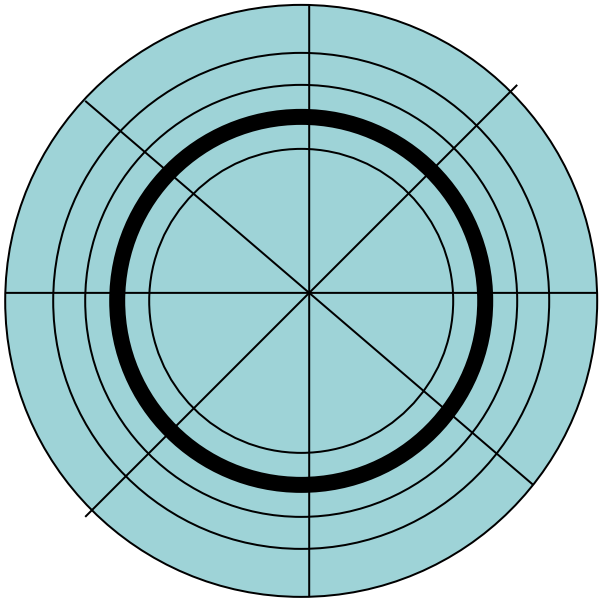
- Recall
  - 1 kB = 1000 bytes 1 KiB = 2^{10} bytes = 1 024 bytes
  - 1 MB = 10^6 bytes 1 MiB = 2^{20} bytes = 1 048 576 bytes
  - 1 GB = 10^9 bytes 1 GiB = 2^{30} bytes $\approx 1.073 \cdot 10^9$ bytes
  - 1 TB = 10^{12} bytes 1 TiB = 2^{40} bytes $\approx 1.100 \cdot 10^{12}$ bytes

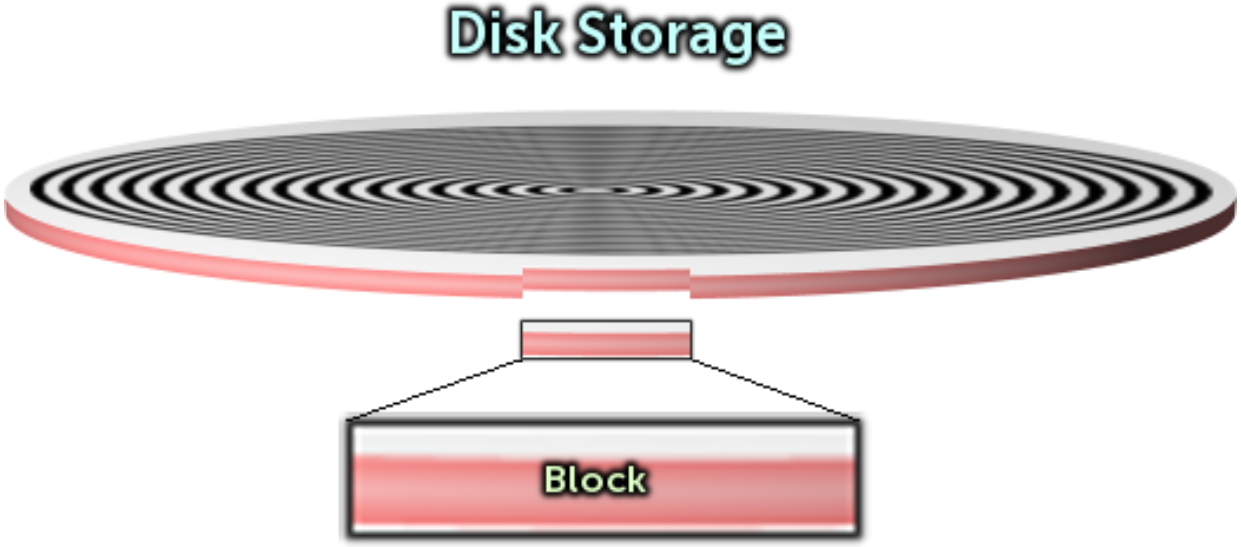
Access Times

- Access time for these devices is variable:
  - Cache ~1 GHz
  - Main memory (SDRAM): ~100 MHz
  - Hard disk (~10 ms seek time) ~100 Hz

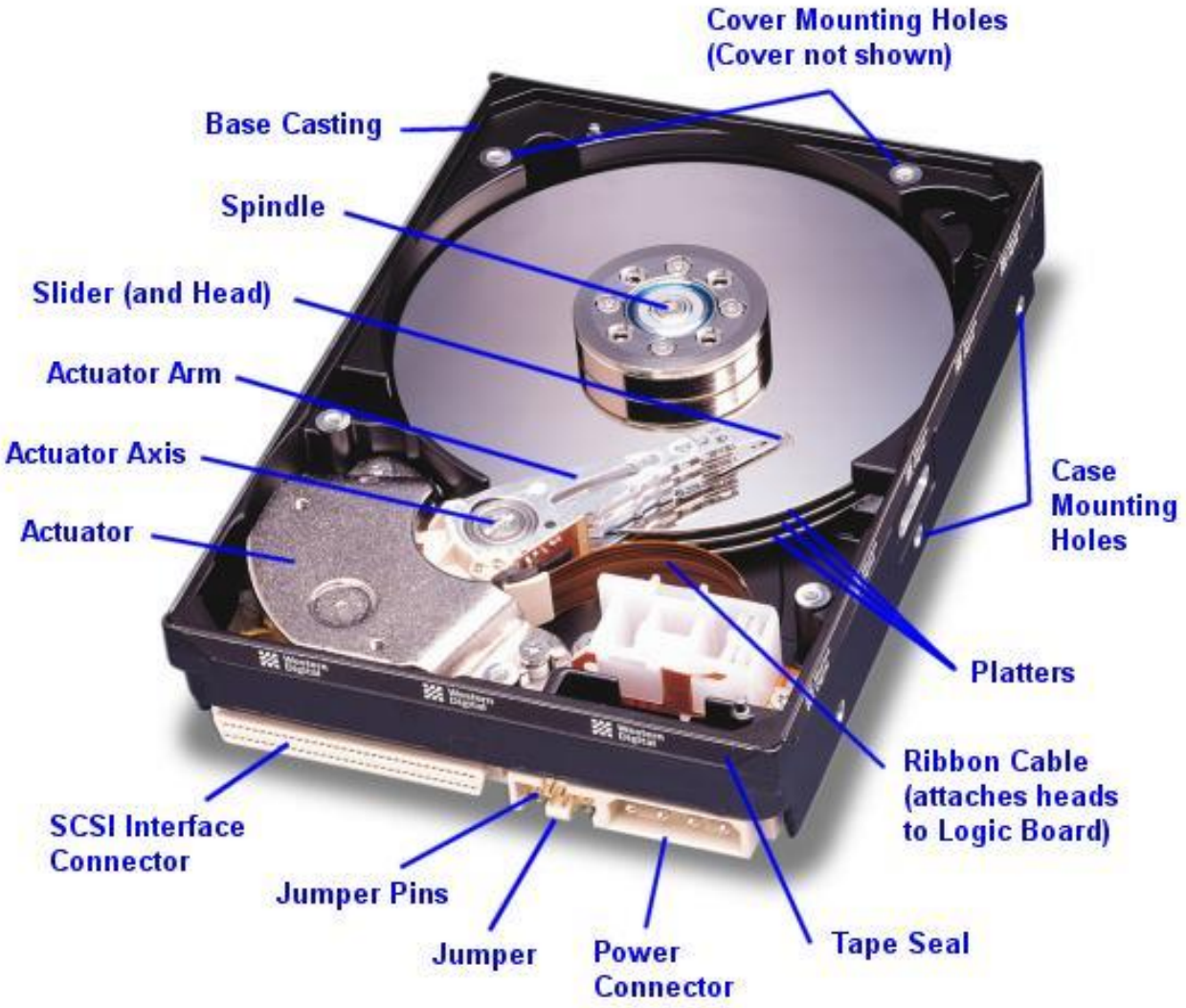
Data Layout on Hard Disk

- Track: one ring
- Sector: one pie-shaped piece
- Block: intersection of a track and a sector





Hard Disk


Cover Mounting Holes
(Cover not shown)
Base Casting
Spindle
Slider (and Head)
Actuator Arm
Actuator Axis
Actuator
Case
Mounting
Holes
Platters
Ribbon Cable
(attaches heads
to Logic Board)
SCSI Interface
Connector
Jumper Pins
Jumper
Power
Connector
Tape Seal

Performance: Disk I/O vs. Processor Speed

- Hard disk is a mechanical device
  - Speed depends on the time required to spin the disk and move head
- Most disks spin at 7200 revolutions per minute (RPM)
  - 120 disk access per second
- On average disk has to spin half way to find the required data
  - Plus the time to move the disk head
- Assuming CPU can process 500 million instructions per second
  - One disk access is worth 4,000,000 instructions
  - Rough calculation but relative speeds are rather clear

Addressing Memory

- Main memory (in general) is byte addressable
  - Each byte has its own address
  - Each byte can be accessed or modified

- To access a bit, you must access the byte containing it

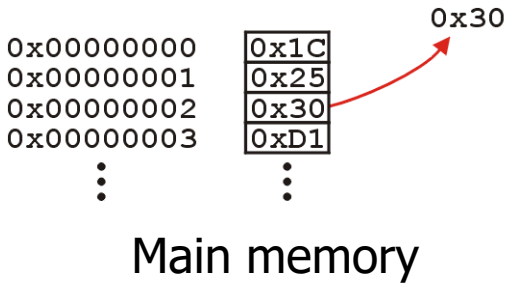
Hard Disk

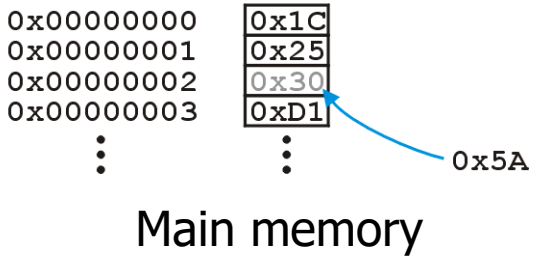
- Hard drives are block addressable
  - The block size is variable
  - Typical size are 512, 2048, 4096, or 8192 bytes
  - When you format your hard drive, you specify the block size

- To access a byte from a block, the hard disk controller must load the entire block containing it in RAM!

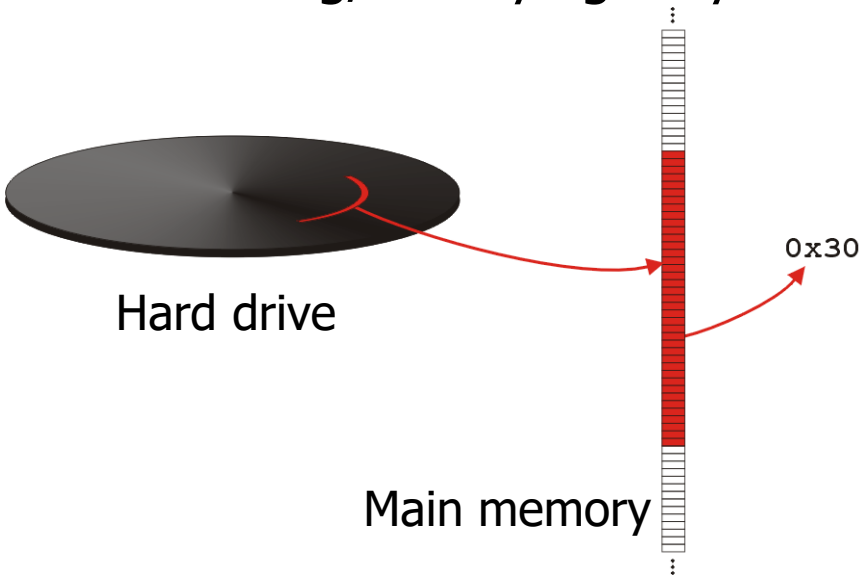
Hard Disk

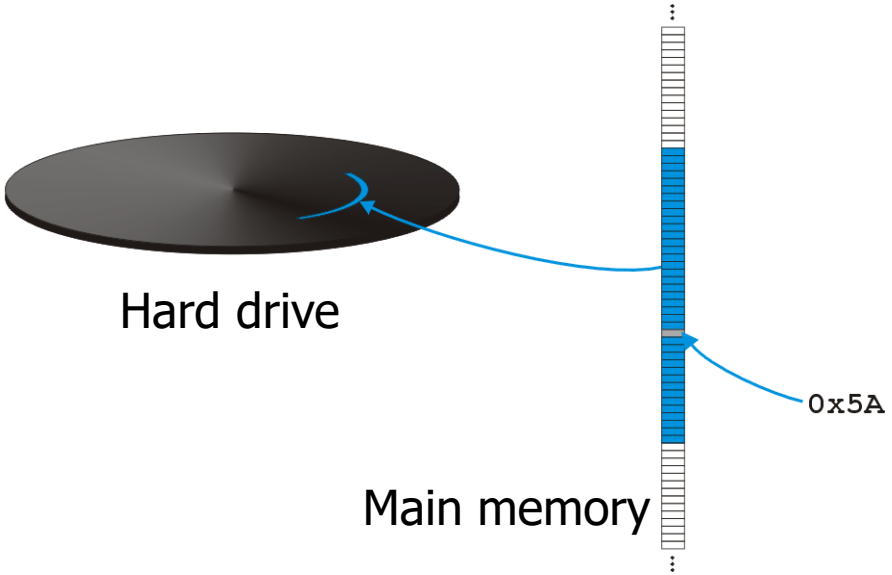
- Accessing/modifying a byte in main memory:





- Accessing/modifying a byte on a hard drive:





Hard Disk: Summary

- Hard drives are very slow
- Divided into blocks
  - Block size is variable and can be specified at format time
- Accessing the entire block costs the same as accessing one byte within the block

Example: AVL Trees

- Number of elements is $n = 2^{30}$ ($\sim 10^9$)
  - Minimum height = 30
  - Maximum height = 43
- Just the pointers requires almost 8 GiB
  - Each node maintains two pointers of 4 bytes each
  - AVL tree has to reside on hard disk
- Very likely that two nodes are not in the same block on hard disk
  - Accessing leaf node requires that 43 blocks be copied from hard disk
  - Assume the root is in main memory
- With 10 ms seek time, this would require approximately 0.43 s
  - Not acceptable

Storing Very Large Trees

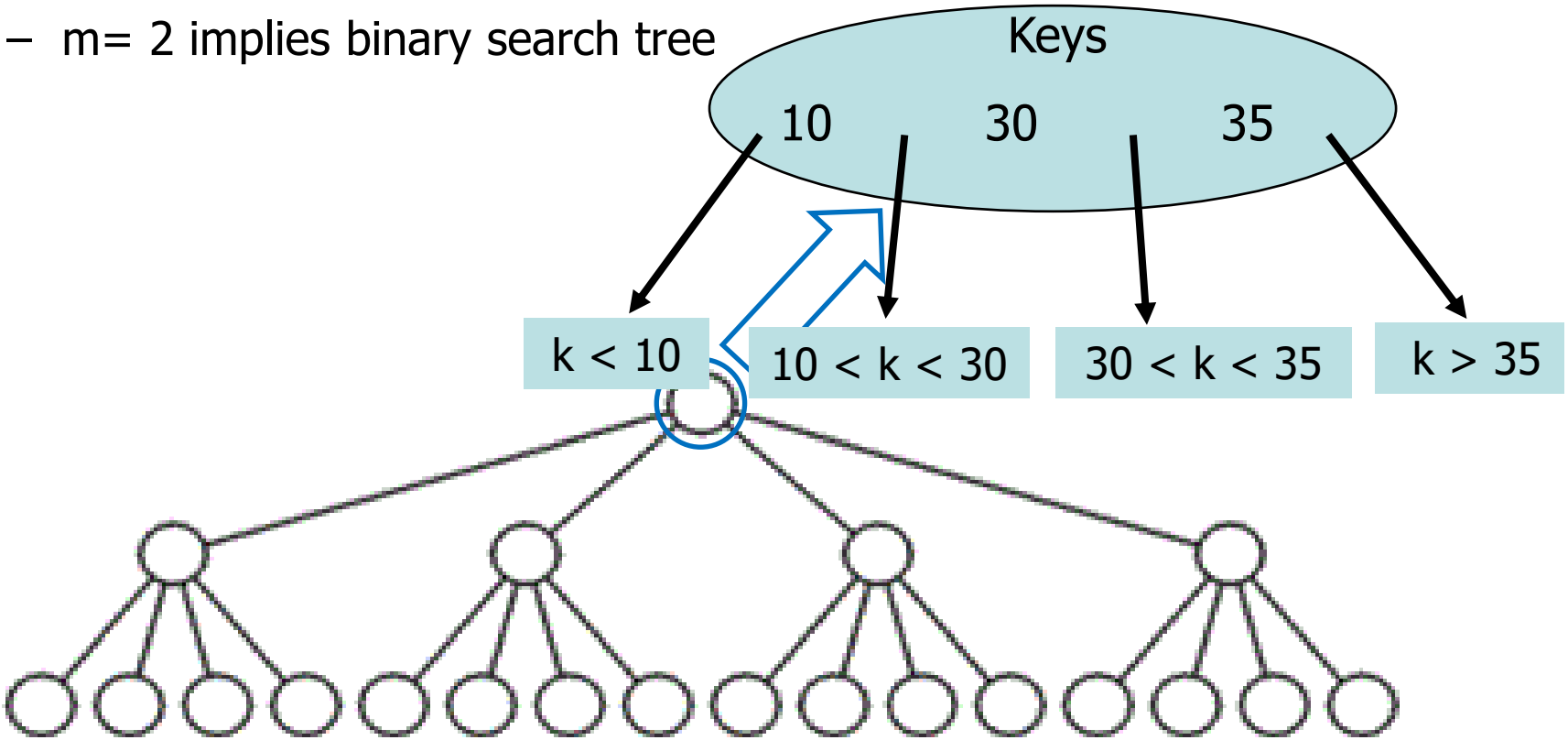
- For many corporate and scientific databases, the search tree must mostly be on disk
  - Visiting node implies accessing disk (i.e., disk I/O)
  - Number of disk accesses dominate the running time
    - Disk I/O is significantly slower than RAM!

- Goal: Decrease Height of Tree
  - Use more branches and thus less height!
  - As branching increases, depth decreases

M-Ary Search Trees

- Allows up to m children for each node
  - Instead of max two for binary trees

- Each node has up to m – 1 keys and m children
  - m= 2 implies binary search tree

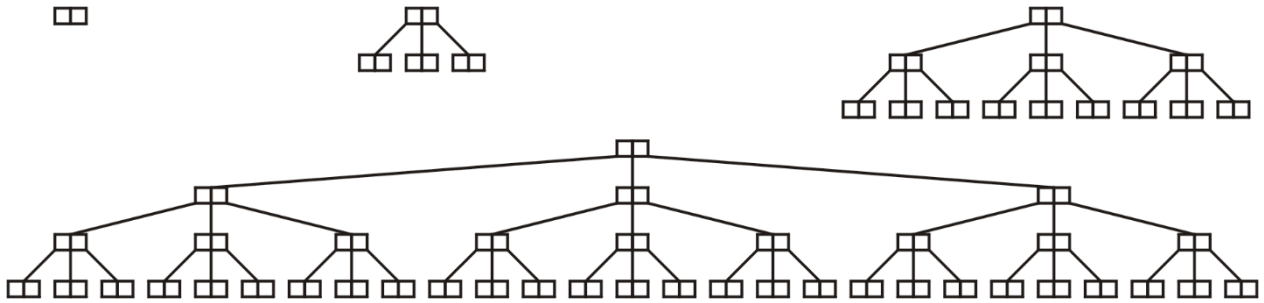


M-Ary Search Trees

- Reduce disk accesses to a very small constant number
  - Such as three or four

- Binary search tree does not work
  - Height of an almost complete binary tree is roughly $Log_2(n)$

- Use search trees that allow m-way branching (m-ary search trees)
  - Height is roughly $Log_m(n)$



3-ary Tree

h	data	Formula
0	2	$3^1 - 1$
1	8	$3^2 - 1$
2	26	$3^3 - 1$
3	80	$3^4 - 1$

B-Tree Definition

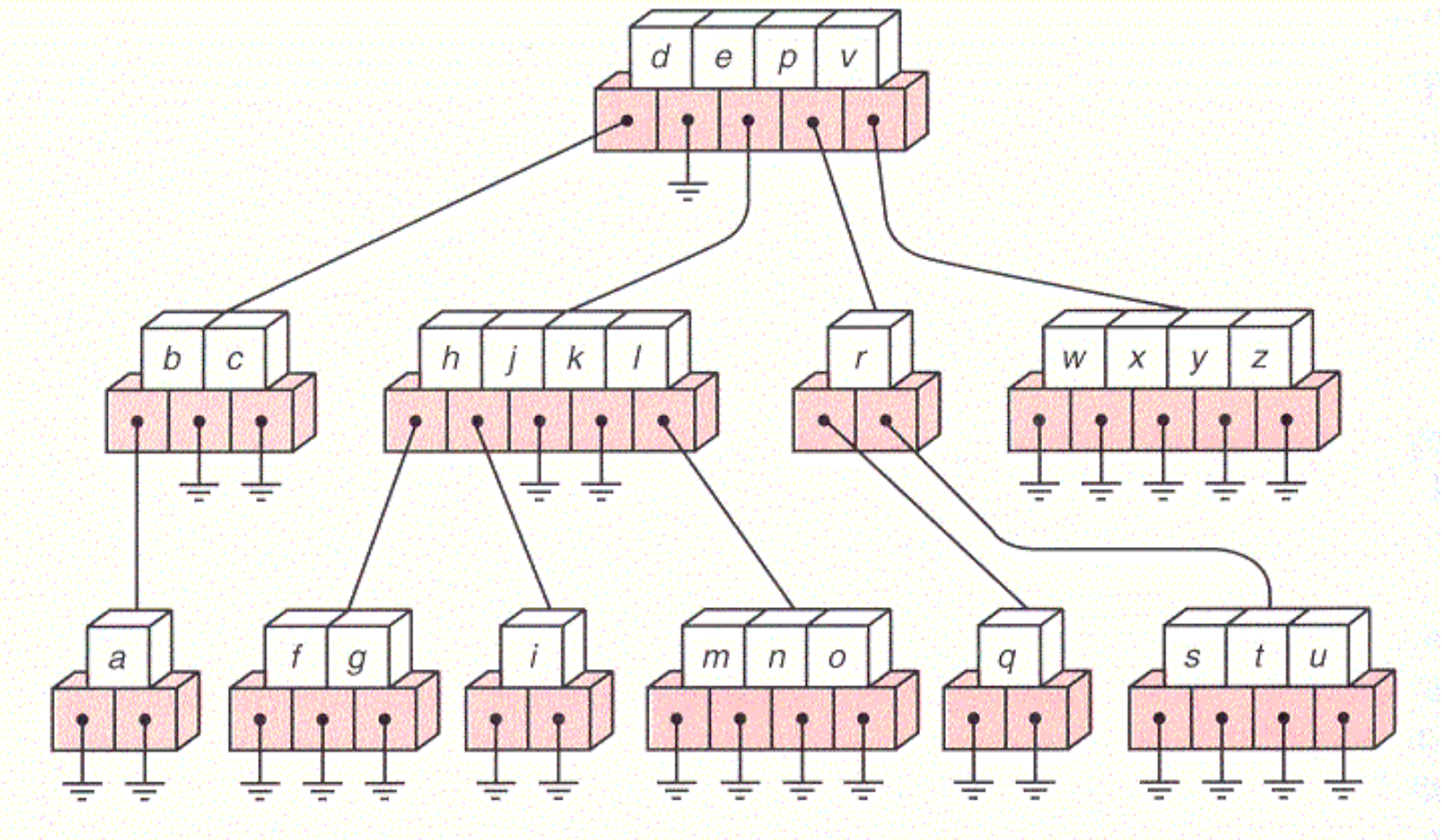
- B-tree of order m is a m-ary tree in which
  - Every node contains no more than $m-1$ keys or m children
    - Keys are partitioned among children to satisfy order property of search tree
  - Root is either a leaf node or has between 2 and m children
  - All non-leaf nodes except root have between $\lceil m/2 \rceil$ and m children
  - All leaves are at the same depth (or level)
  - Data items are stored at the leaves (B+ Trees)

B-Tree: Summary

	Root node	Non-root node
Minimum number of keys	**1**	**$\lceil m / 2 \rceil - 1$**
Minimum number of non-empty subtrees	2	$\lceil m / 2 \rceil$
Maximum number of keys	**m - 1**	**m – 1**
Maximum number of non-empty subtrees	m	m

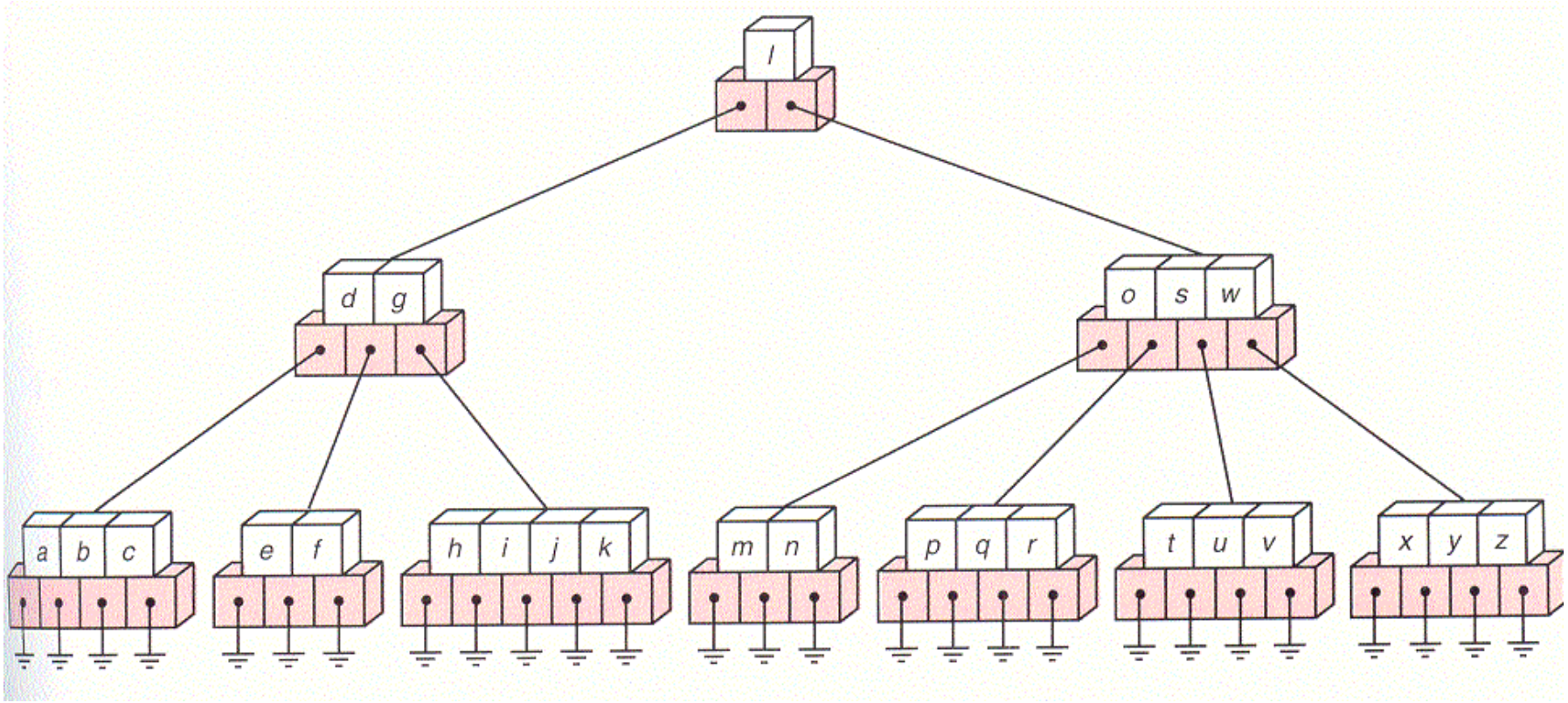
Example

- Is it B-tree of order 5?
  - All non-root nodes do not have between 3 and 5 children
  - All leaves are not on the same level

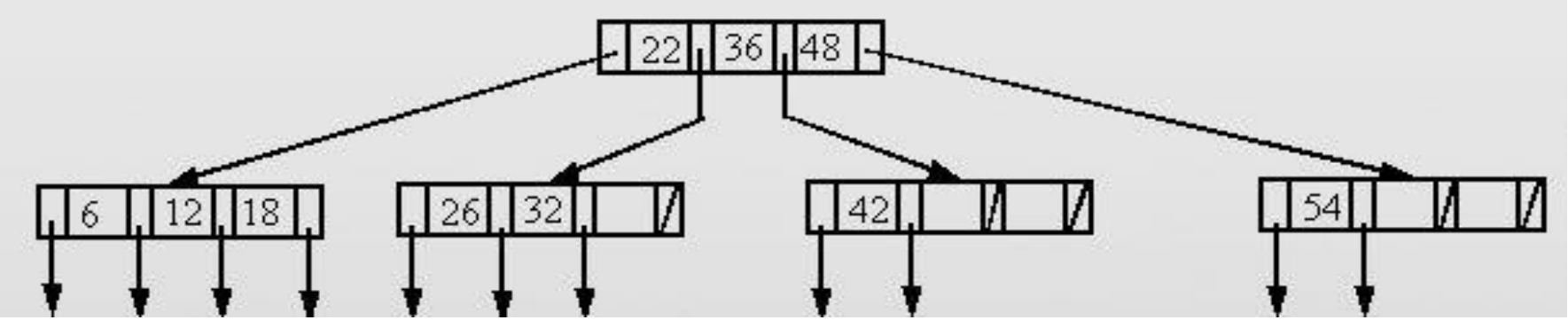


Example

- B-tree of order 5

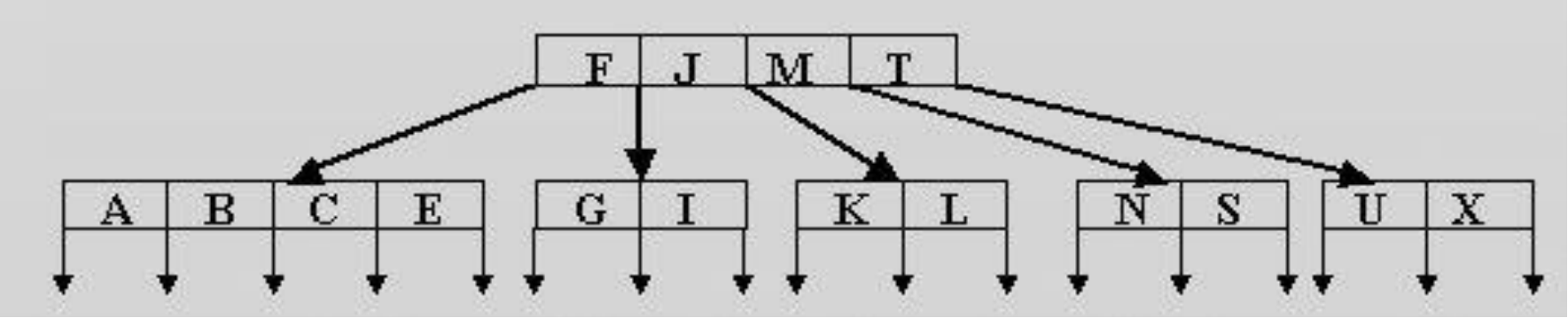


Example



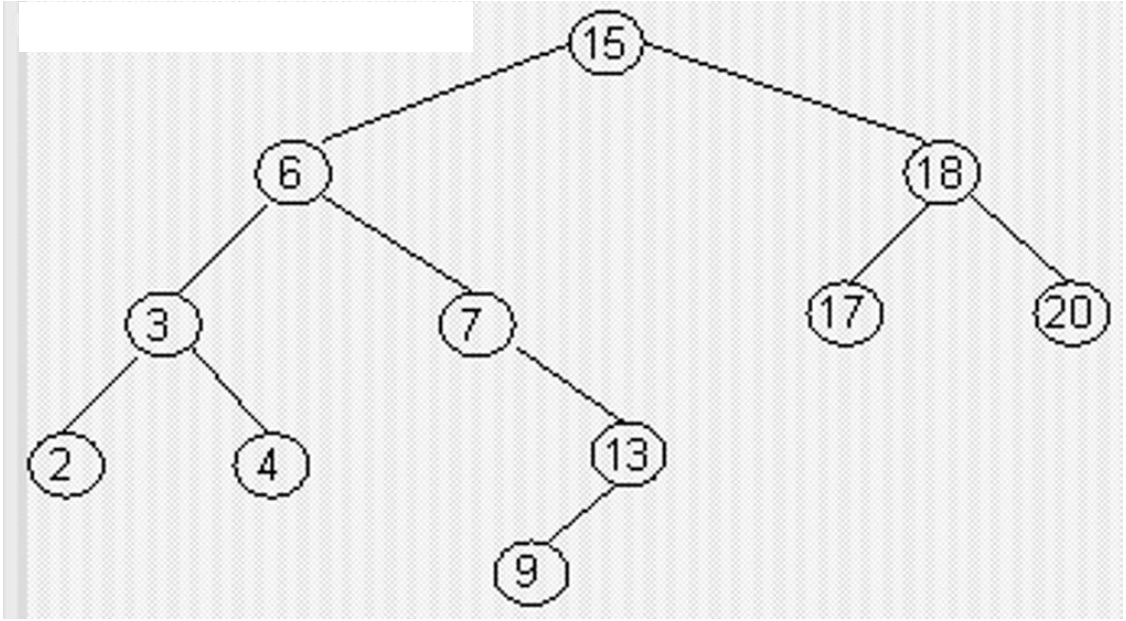
- Is it a B-tree?
  - Yes
- What is the order of B-Tree?
  - 4

Example



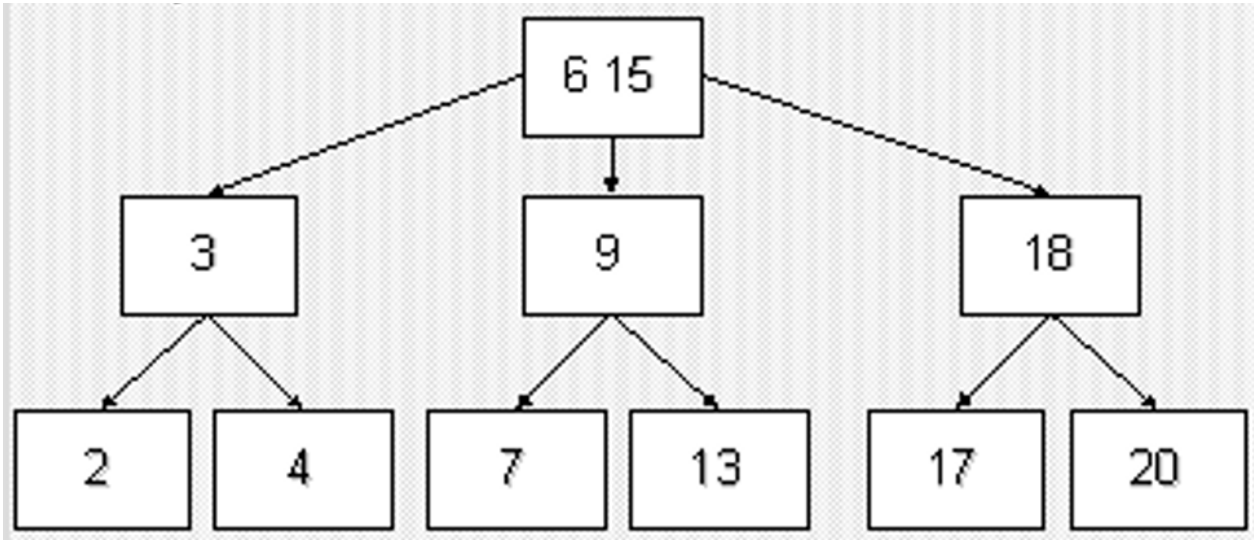
- Is it B-tree?
  - Yes
- What is the order of B-Tree?
  - 5

B-Tree vs. Binary Search Tree (BST)



Height h = 4
2-leaves at depth 2
2-leaves at depth 3
1-leaf at depth 4

Order = 3
Height h = 2
6-leaves at depth 2



B-Tree Properties

- B-Tree is a rooted tree (`root[T]`) with the following properties
  - Every node x has the following fields
    - `n[x]` – number of keys currently stored in x
    - `n[x]` keys – stored in ascending/increasing order
      - » $key_1[x] \le key_2[x] \le \ldots \le key_{n[x]}[x]$
    - `Leaf[x]` – boolean value that is `TRUE` if x is leaf
  - An internal node x contains `n[x]+1` pointers to its children
    - Leaf node contains no children
  - The keys $key_i[x]$ separate the range of keys stored in each subtree
    - If k_1 is any key stored in the subtree with root $c_1[x]$, then
    - $k_1 \le key_1[x] \le k_2 \le key_2[x] \le \ldots key_{n[x]}[x] \le k_{n[x]+1}$
  - Each leaf has the same depth, which is the height `h` of B-tree

Example: B-Tree of Order 5

- All internal nodes except root have at least $\lceil m/2 \rceil = \lceil 5/2 \rceil = 3$ children
  - Every non-root node have at least 2 keys

- Maximum number of children that a node can have is $m=5$
  - Maximum number of keys are $m-1 = 4$

Height Of B-Tree

- Notation
  - Let h be the height of B-tree
  - Let $n \geq 1$ be the number of entries in B-tree
  - Let m be the maximum number of children a node can have
  - Let $t = \lceil m/2 \rceil$ be minimum number of children a node can have

- Best case height of B-Tree
  - $\mathbf{h = \lceil \log_m (n+1) \rceil - 1}$
- Worst case height of B-Tree
  - $\mathbf{h \leq floor[\ \log_t (n+1/2)]}$

- Worst case of a B-Tree provides an upper bound on the number of disk accesses

Example

- Let
  - n= 2,000,000 (2 Million) entries
  - Maximum number of children = 100,
  - Minimum number of children = 50

- Worst case height h of B-Tree = 3

- Height h of binary tree ~ 20

Operations on B-Tree

- Creating an empty B-Tree
- Searching a B-Tree
- Inserting a Key into B-Tree
- Deleting a Key from B-Tree

Search in a B-Tree

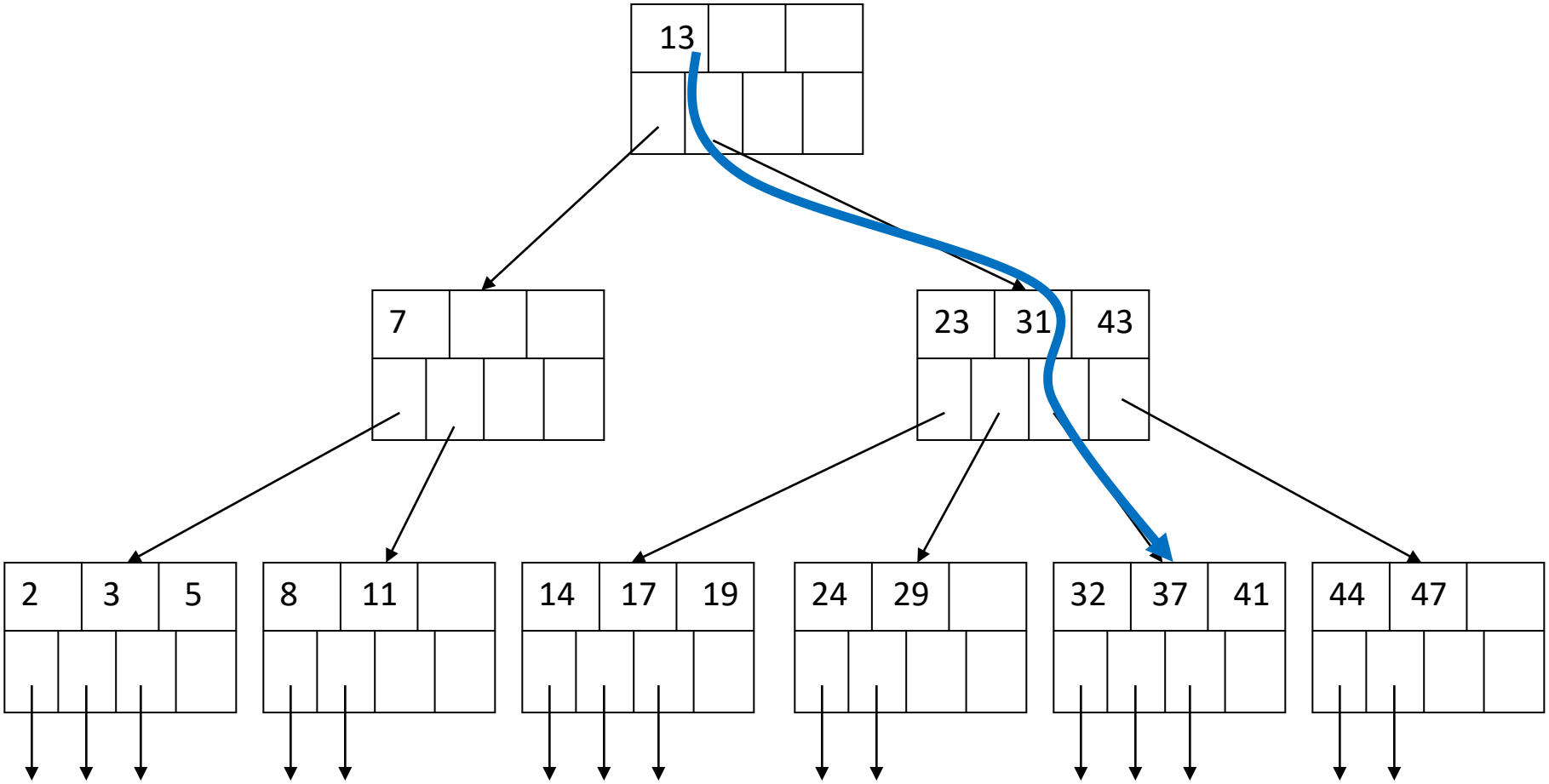
Searching in B-Tree

- Similar to searching in a BST
  - BST makes a binary or "two way" decision at each node
  - B-Tree makes a multi way branching decision according to number of children

- Each internal node `x` makes a `(n[x]+1)`-way branching decision

Example: Searching in B-Tree

- Find a record with search key 40
  - Start at the root



Searching in B-Tree

- B-trees are used for queries where a range of values are searched
  - Range queries

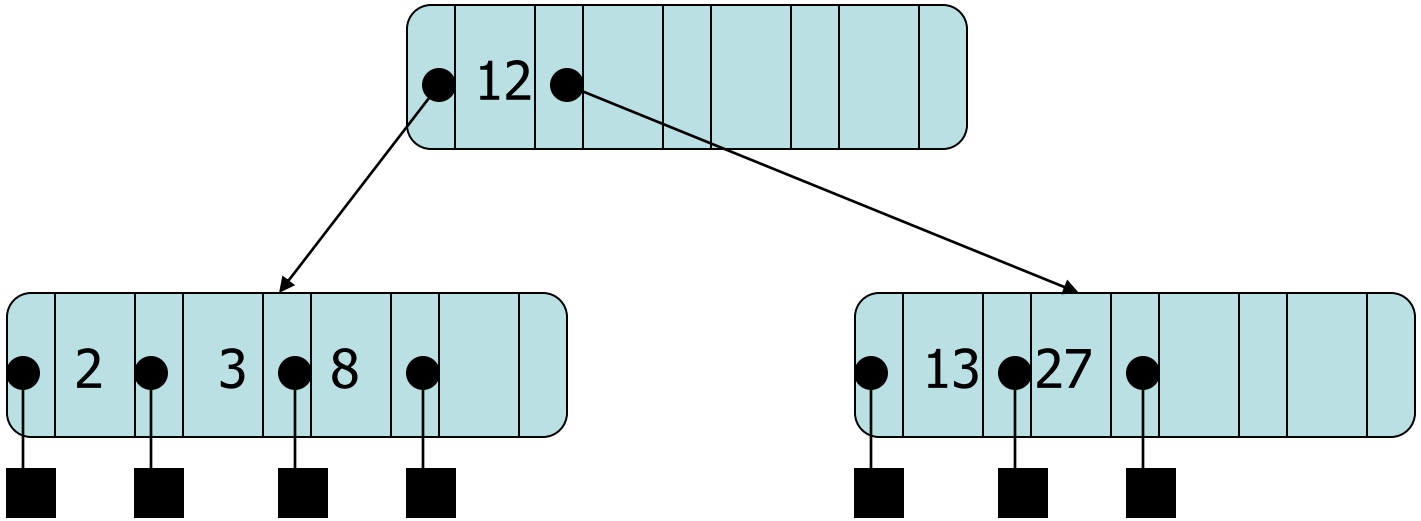
- Example: `SELECT * FROM R WHERE R.k ≥ 10 AND R.k ≤ 25;`

Insertion in a B-Tree

Insertion in B-Tree

- Insertion in a B-tree in different and complicated than BST
  - First, appropriate node for the key must be located using an algorithm similar to B-Tree Search
  - Second, key must be inserted into the node
  - Third, check whether the node is full
    - If the node is not full no special action is required
    - Else split operation is performed on the node

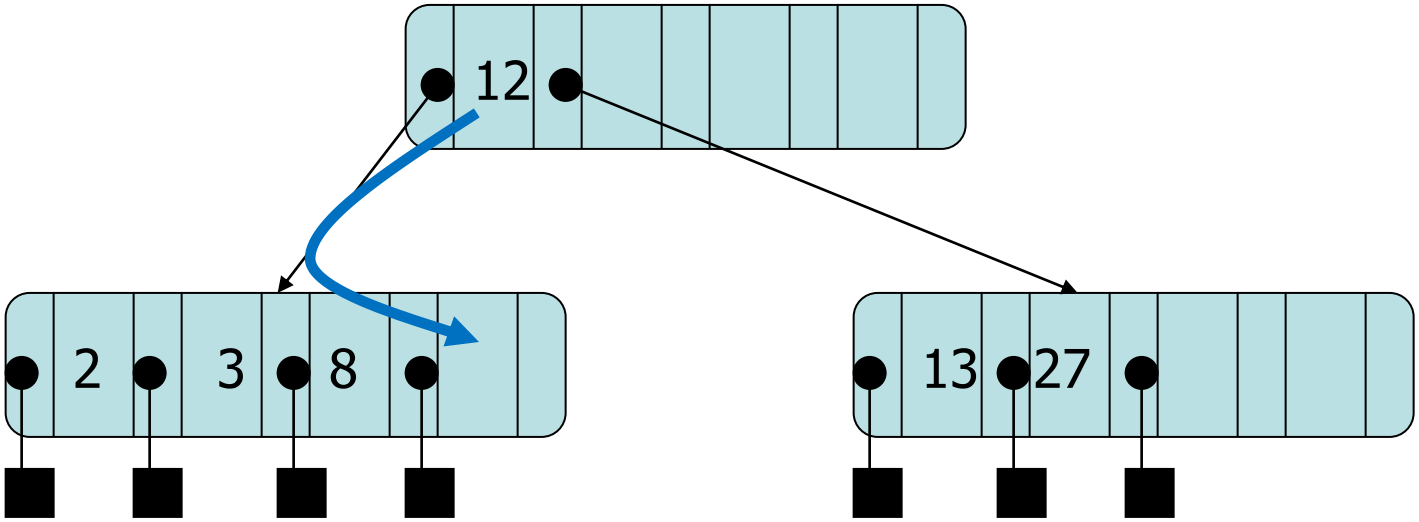
Example: B-Tree with m= 5



- The root has between 2 and m children
- Each non-root internal node has between $\lceil m/2 \rceil$ and m children
- All leaf nodes are at the same depth

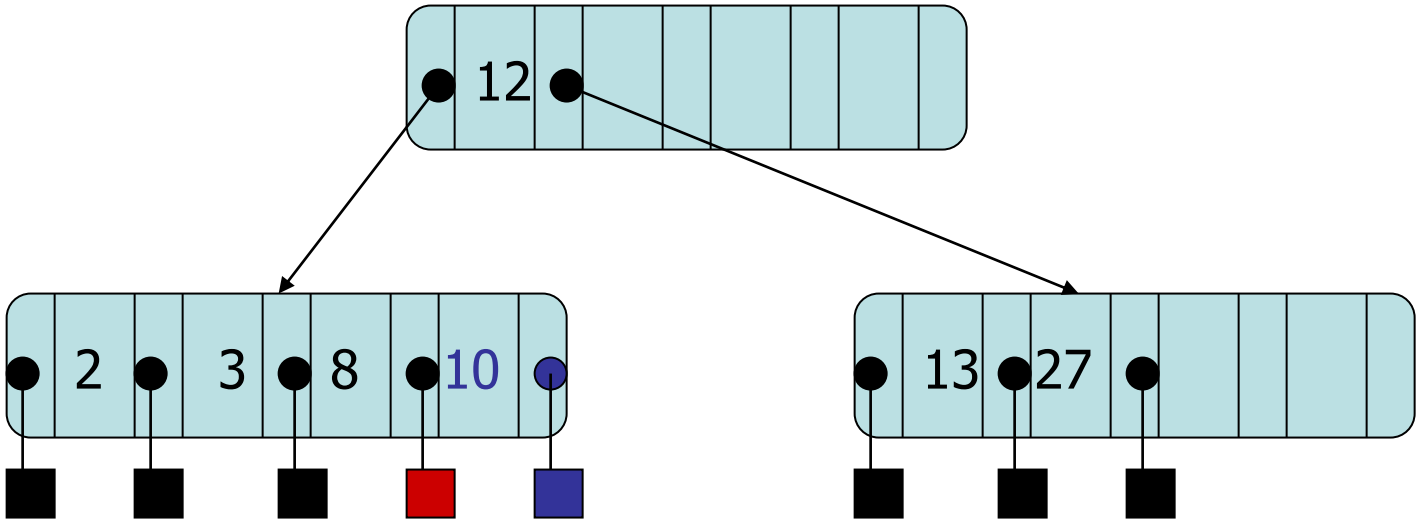
Example: B-Tree with m= 5

- Insert 10
  - Find the location for 10 by following a path from the root using the stored key values to guide the search
  - The search falls out the tree at the 4th child of the left child of the root



Example: B-Tree with m= 5

- Insert 10
  - Find the location for 10 by following a path from the root using the stored key values to guide the search
  - The search falls out the tree at the 4th child of the left child of the root
  - The node has room for the new element, so 10 is stored

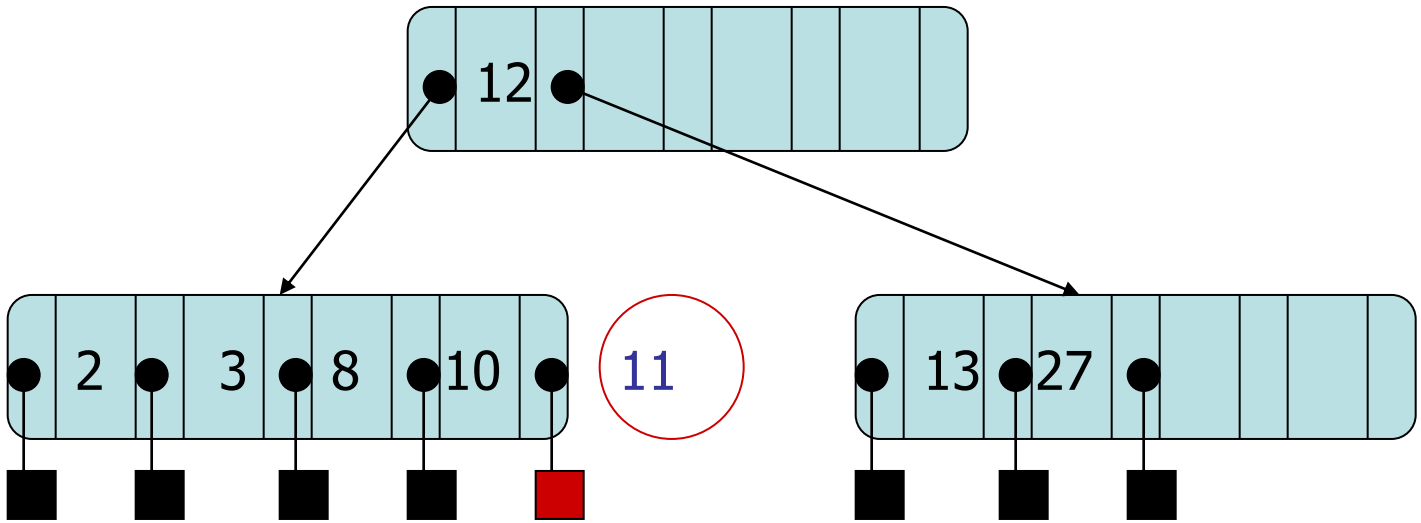


Splitting a Node in B-Tree

- If a node becomes full, it is necessary to perform a split operation
- A full node y (having $m-1$ keys) splits around its median key
  - Creates two nodes having $\lceil m/2 \rceil - 1$ keys each
- The median key moves up into y's parent
  - If the y's parent node is full, it has to be split too
  - This process may repeat all the way up to the root
- If y has no parent (i.e., root), then the tree grows in height by one
  - So splitting is the mean by which B-Tree grows

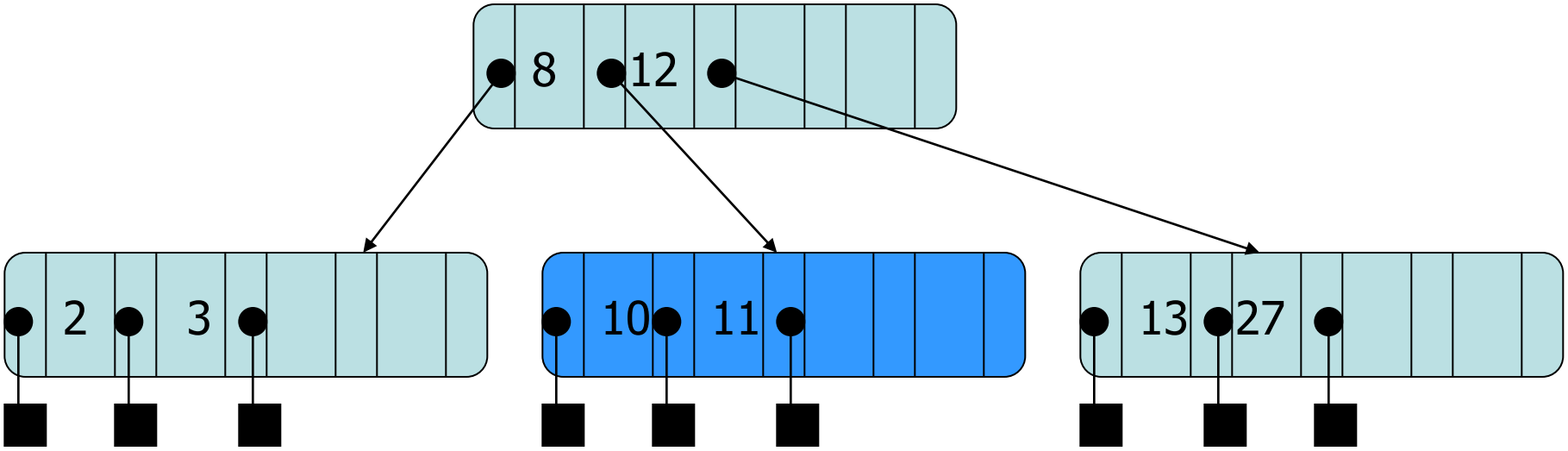
Example: B-Tree with m= 5

- Insert 11
  - The search falls out the tree at child to the right of key 10
  - No more space in the left child of the root to hold 11
    - The left child must be split



Example: B-Tree with m= 5

- Splitting the node
  - The `m + 1` children are divided evenly between the old & new nodes
  - The parent gets one new child
    - If the parent become overfull, then it, too, will have to be split



Splitting a Node in B-Tree

- Splitting the node results in moving one key to the parent node
- What if the parent node is full?
  - Parent has to be split too
- Splitting process may repeat all the way up to the root and may require splitting the root node
- In general, splitting approach requires two passes
  - First pass locates the node where the key should be inserted
  - Second pass performs any required splits on the ancestor nodes

Splitting a Node in B-Tree: Avoiding Second Pass

- Each access to a node may correspond to a costly disk I/O
- Desirable to avoid the second pass by ensuring that the parent node is never full
- Idea: Algorithm splits any full nodes encountered while descending the tree (during first phase)
- Is there a problem?

Splitting a Node in B-Tree

- This approach may result in unnecessary split operations
- Guarantees that the parent never needs to be split
- Eliminates the need for a second pass up the tree
- What is the penalty?
- Since a split runs in linear time, it has little effect on the running time of B-Tree-Insert

Example: Insertion in B-Tree (Order 5)

- Insert the following elements in B-Tree
  - C N G A H E K Q M F W L T Z P R X Y D S

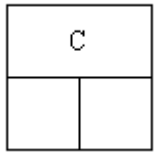
- Order 5 indicates
  - A node can have a maximum of 5 children and 4 keys
  - All nodes other than the root must have a minimum of 2 keys

- Height of B-Tree grows with splitting only

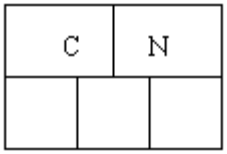
Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

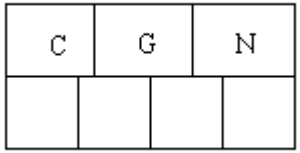
- Insert C



- Insert N



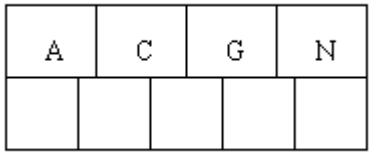
- Insert G



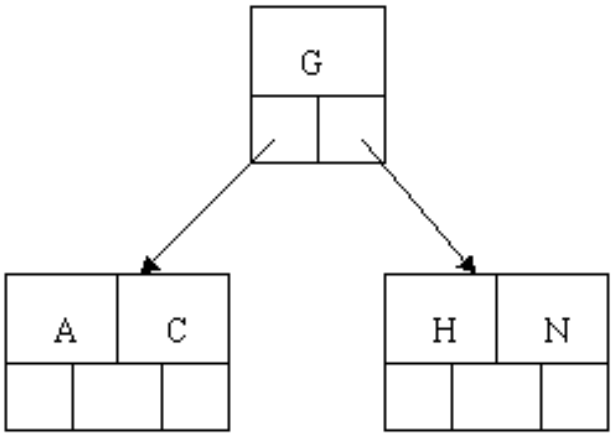
Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

- Insert A



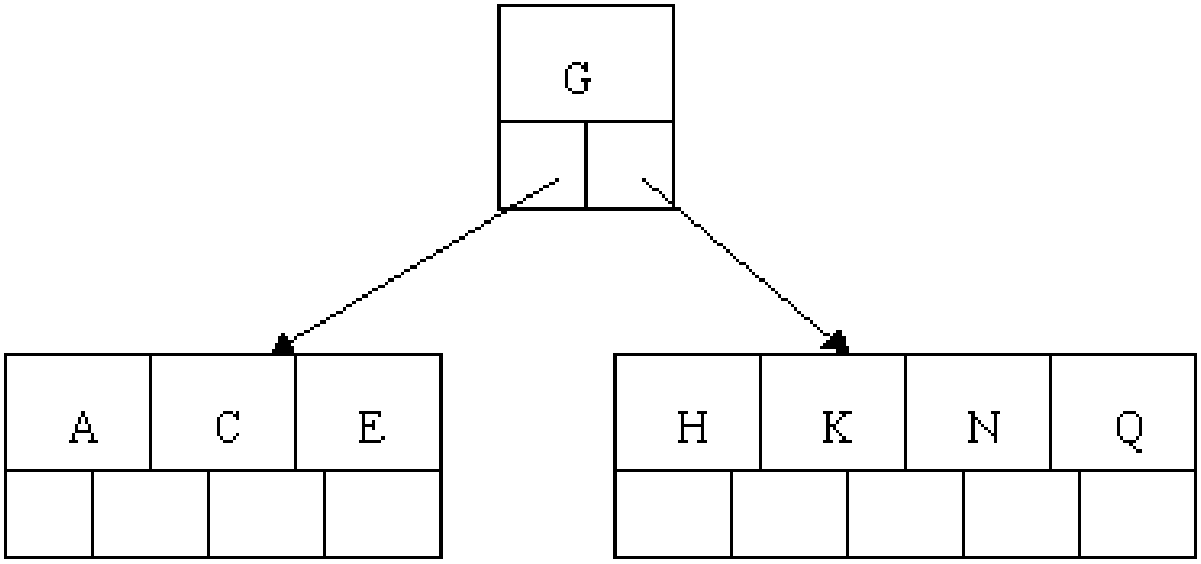
- Insert H
  - Node is already full we have to split it



Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

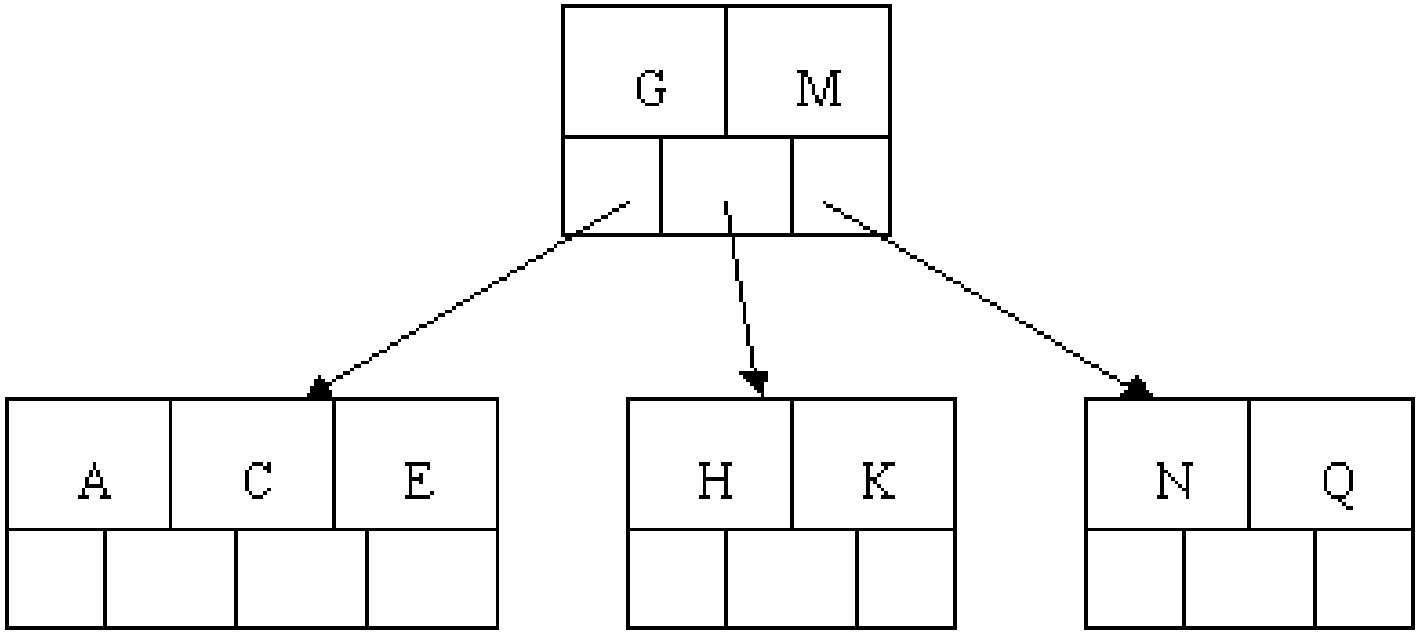
- Insert E, K, Q
  - Require no splitting



Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

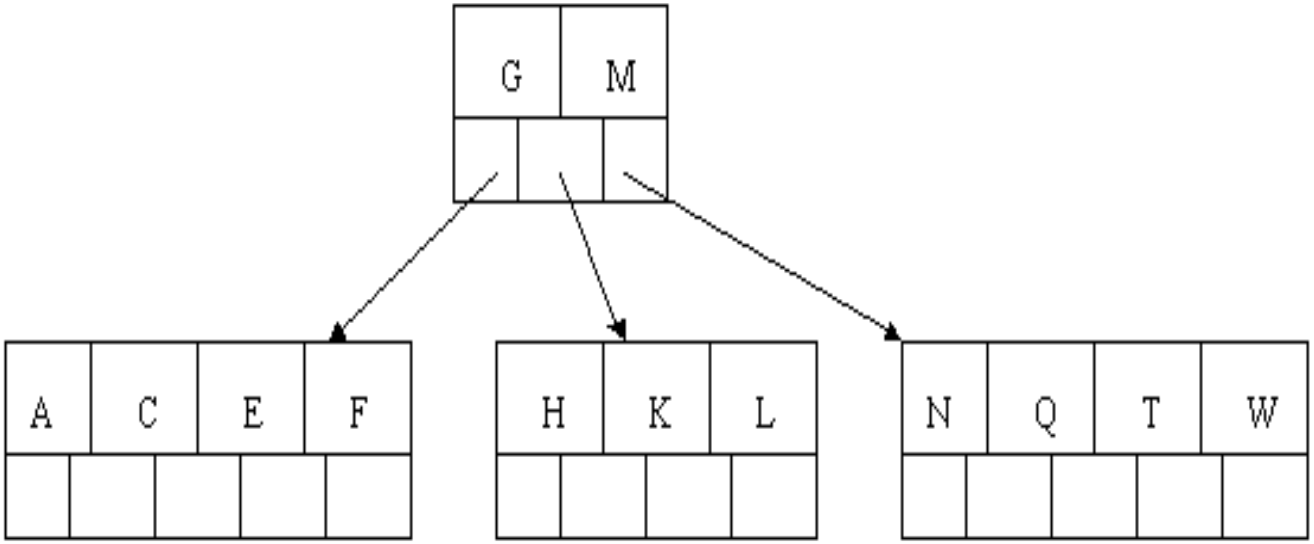
- Insert M
  - M happens to be a median value so moved up the parent node



Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

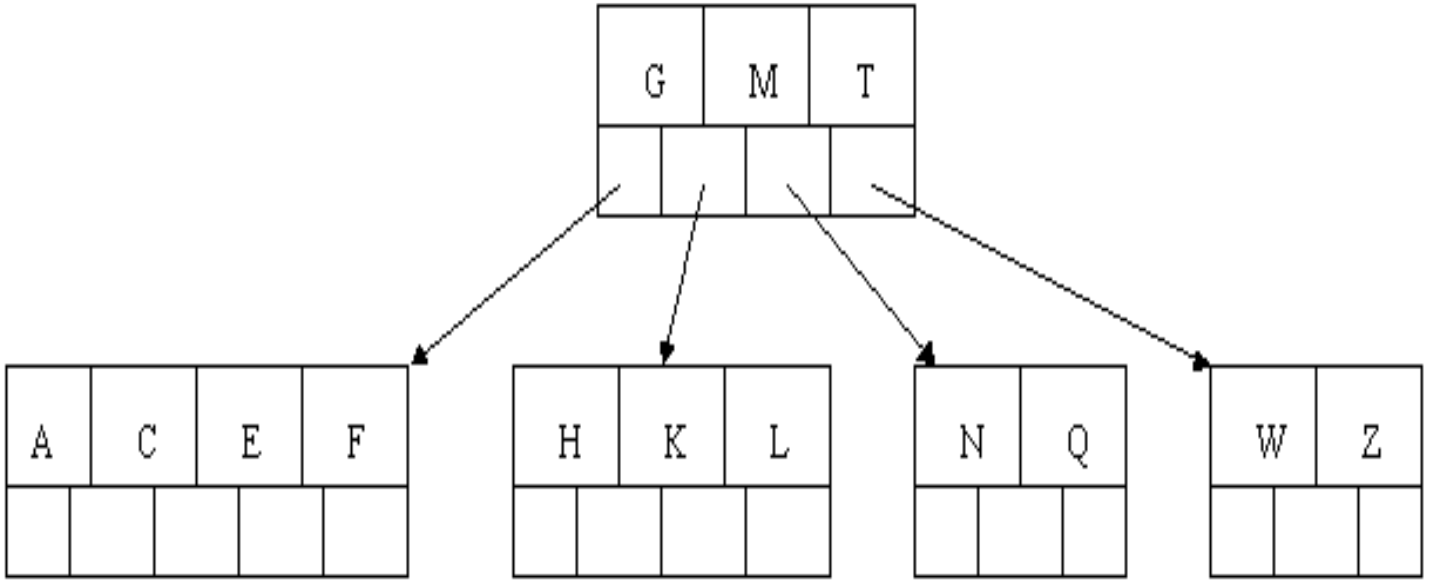
- Insert F, W, L and T
  - No split requires



Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

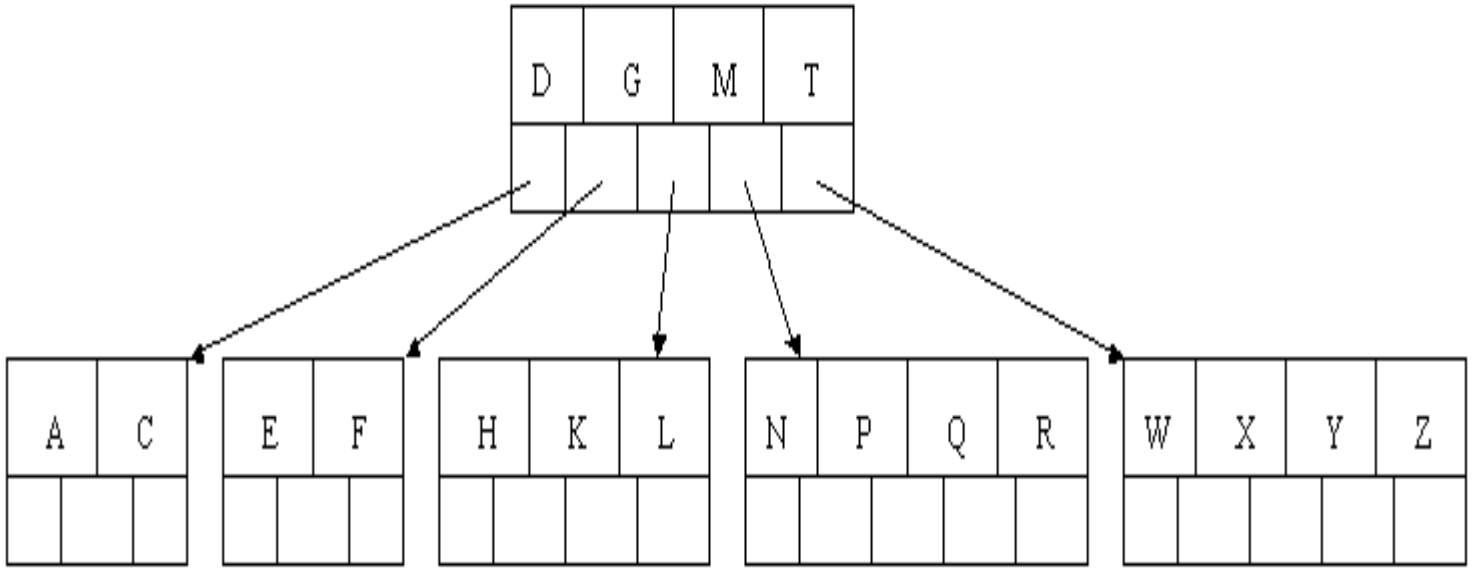
- Insert Z
  - Rightmost leaf must be split
  - Median value T is moved up into the parent



Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

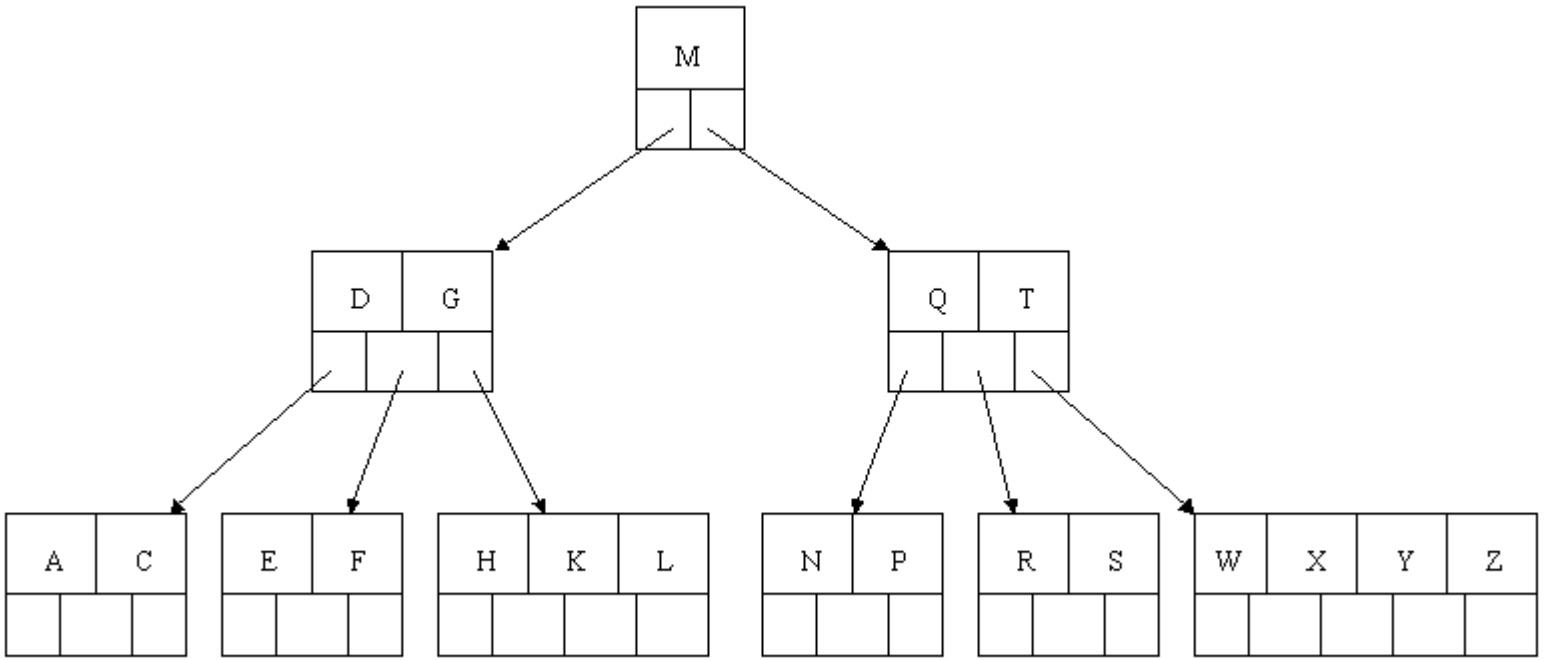
- Insert D
  - Leftmost leaf must be split



Example: Insertion in B-Tree (Order 5)

C N G A H E K Q M F W L T Z P R X Y D S

- Insert S
  - 4th Node splits
  - Parent node is full so it splits



Deletion in a B-Tree

Deletion in B-Trees

- In general, we can either
  - Locate and delete the item, then restructure the tree
  - Do a single pass down the tree, restructure the tree before-hand
- We will focus on the first approach

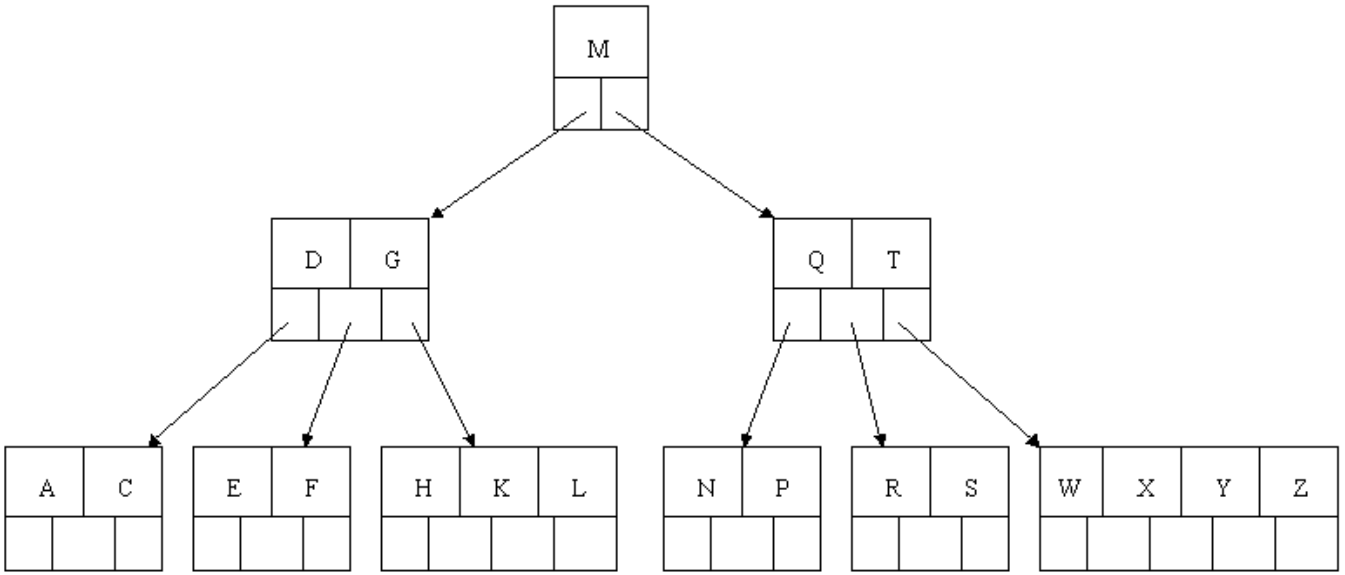
Deletion in B-Trees: Cases

- Case 1: Deletion of a key value from a leaf node
  - Case 1.a: Removal of key leads to equal or more than $\lceil m/2 \rceil - 1$ keys
    - i.e., $\geq \lceil m/2 \rceil - 1$
  - Case 1.b: Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
    - i.e., $< \lceil m/2 \rceil - 1$
- Case 2: Deletion of a key value from a non-leaf node

Deletion From Leaf Node: Case 1.a

- Removal of key leads to equal or more than $\lceil m/2 \rceil - 1$ keys
  - Does not violate the requirement of minimum number of keys

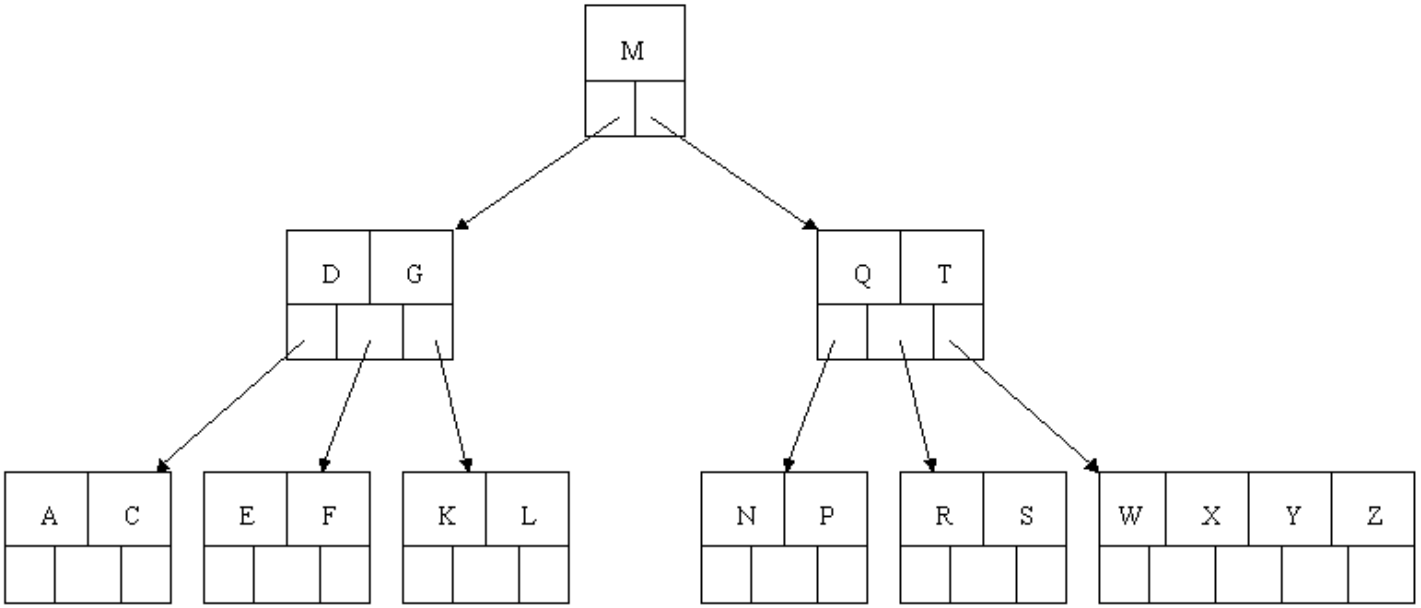
- Example: Delete H from B-Tree of order 5



Deletion From Leaf Node: Case 1.a

- Removal of key leads to equal or more than $\lceil m/2 \rceil - 1$ keys
  - Does not violate the requirements of minimum number of keys

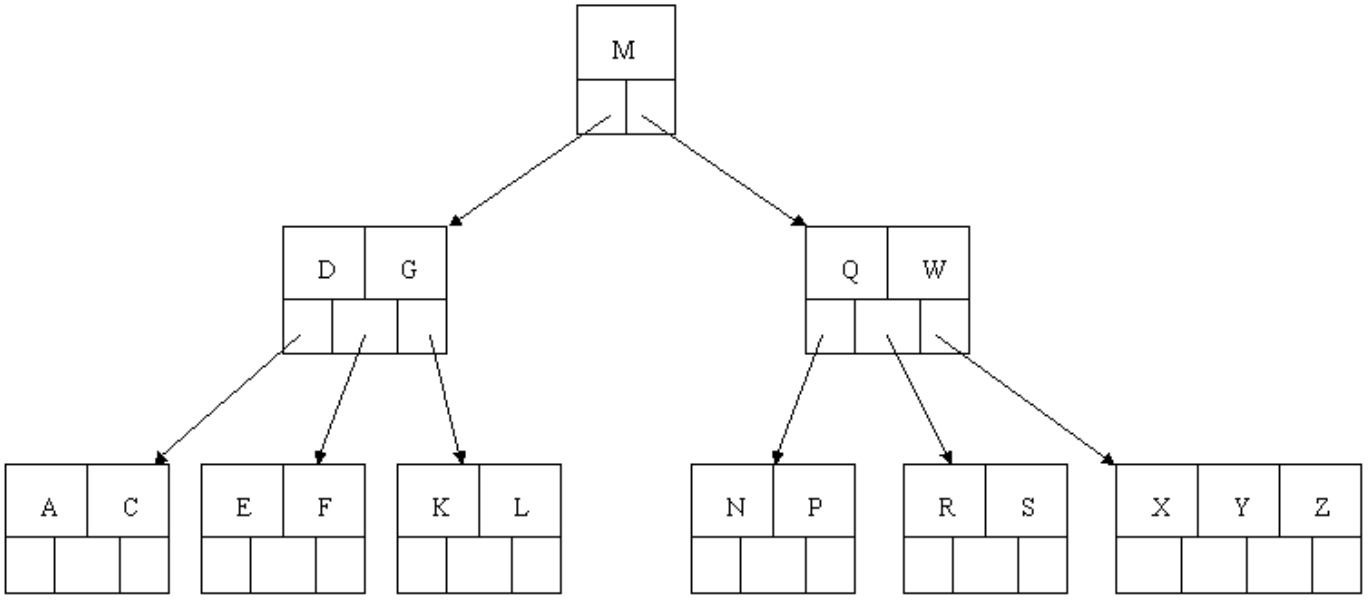
- Example: Delete H from B-Tree of order 5
  - The node still contains $\lceil m/2 \rceil - 1$ keys, i.e., 2



Deletion From Leaf Node: Case 1.b

- Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
  - Violation of the requirement pertaining to minimum number of keys

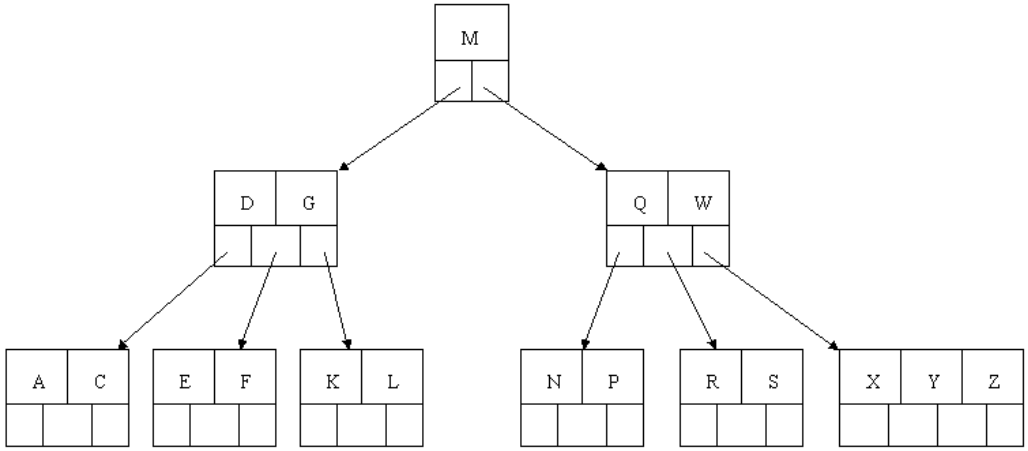
- Example: Delete R from B-Tree of order 5



Deletion From Leaf Node: Case 1.b

- Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
  - Violation of the requirement pertaining to minimum number of keys

- Three situations may be possible in this case 1.b
  1. The nearest right sibling contains more than $\lceil m/2 \rceil - 1$ key values
  2. The nearest left sibling contains more than $\lceil m/2 \rceil - 1$ key values
  3. Neither the nearest left sibling nor the right sibling contain more than $\lceil m/2 \rceil - 1$ key values



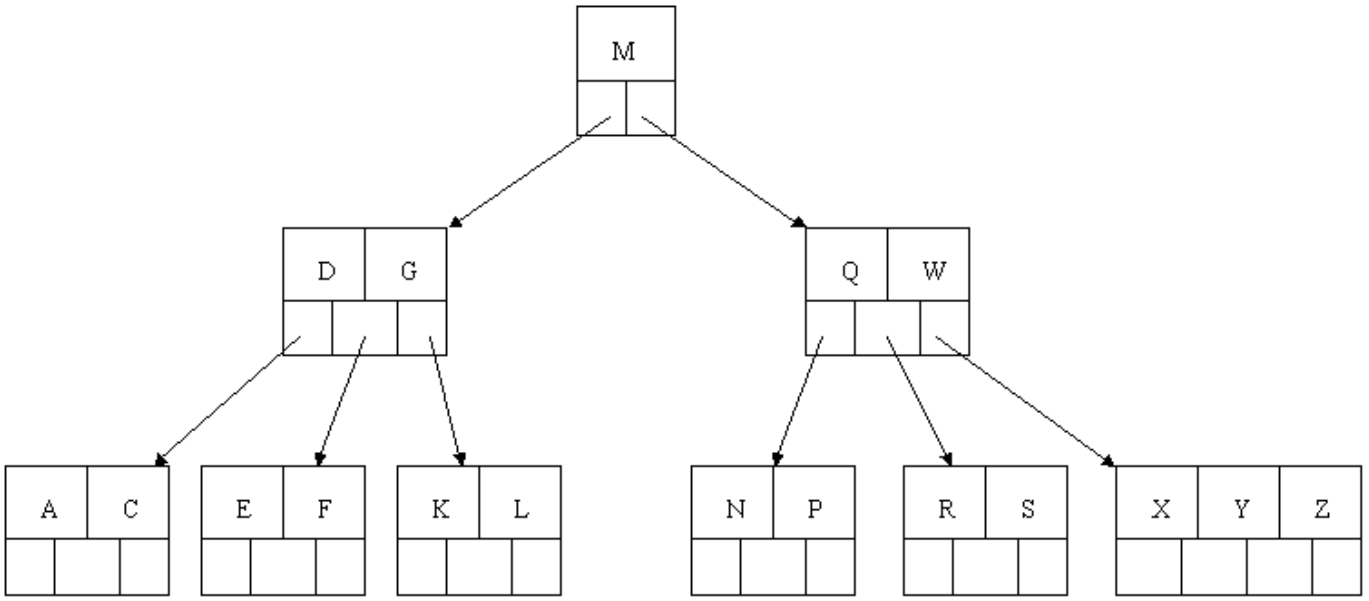
Deletion From Leaf Node: Case 1.b (Situation 1)

- Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
  - Right sibling contains more than minimum number of keys (situation 1)

- Approach
  - Add the key from the parent to the end of the deficient node
  - Move the first element of the right sibling to the parent
  - Adjust the child of right sibling
    - Append first child of the right sibling as the last child of the deficient node

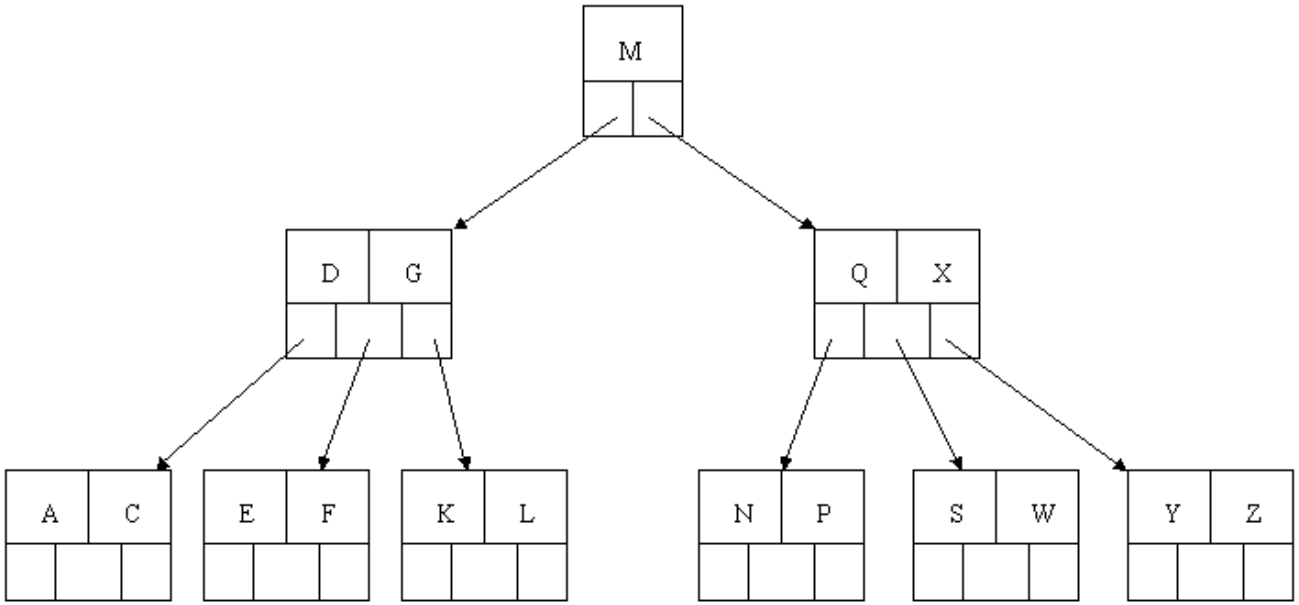
Example: Delete R from B-Tree of order 5

- Deletion results in a node with only one key
  - Not acceptable for a B-tree of order 5
- Sibling to the right has an extra key
  - Key W is moved down as last key from the parent
    - Key S is moved over so that W can be inserted in its proper place
  - Key X from right sibling is moved up to the parent



Example: Delete R from B-Tree of order 5

- Deletion results in a node with only one key
  - Not acceptable for a B-tree of order 5
- Sibling to the right has an extra key
  - Key W is moved down as last key from the parent
    - Key S is moved over so that W can be inserted in its proper place
  - Key X from right sibling is moved up to the parent



Deletion From Leaf Node: Case 1.b (Situation 2)

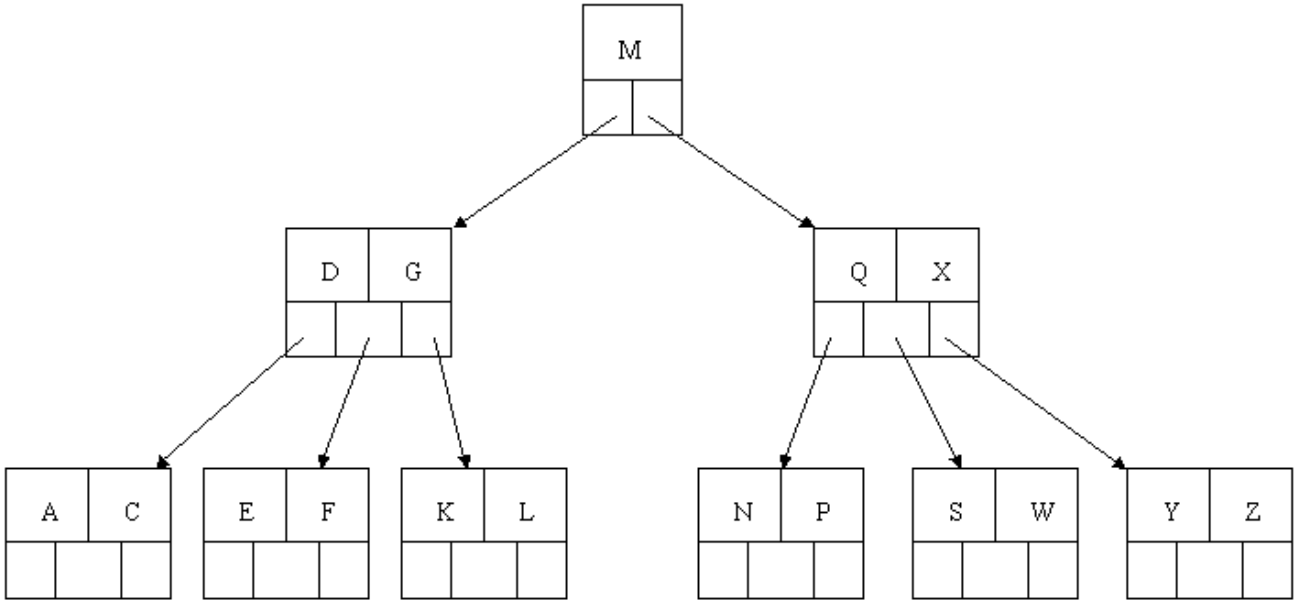
- Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
  - Left sibling contains more than minimum number of keys (situation 2)

- Approach
  - Add the key from the parent to the end of the deficient node
  - Move the last element of the left sibling to the parent
  - Adjust the child of left sibling
    - Insert last child of the left sibling as the first child of the deficient node

Deletion From Leaf Node: Case 1.b (Situation 3)

- Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
  - Neither the left nor the right sibling contains more than $\lceil m/2 \rceil - 1$ keys (situation 3)

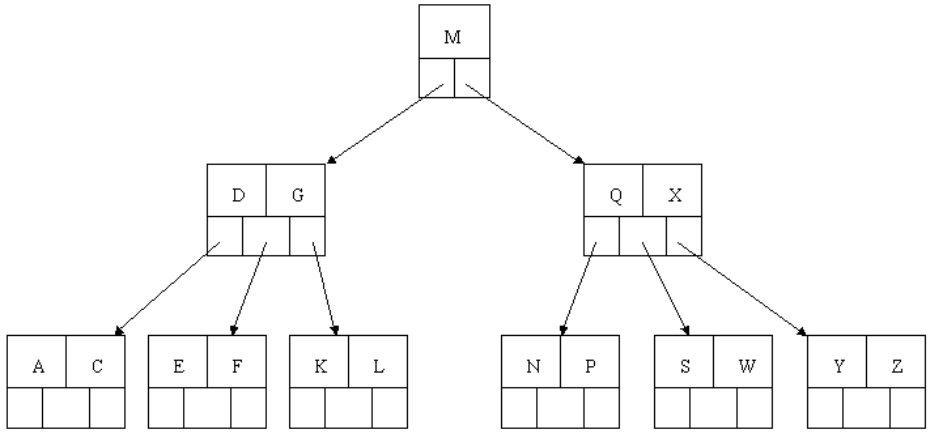
- Example: Delete E from B-Tree of order 5



Deletion From Leaf Node: Case 1.b (Situation 3)

- Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
  - Neither the left nor the right sibling contains more than $\lceil m/2 \rceil - 1$ keys (situation 3)

- Approach
  - Create a new node with the following keys
    - All the key values from the deficient node
    - All the key values from one of its siblings
    - Key from the parent between the two combined sibling nodes



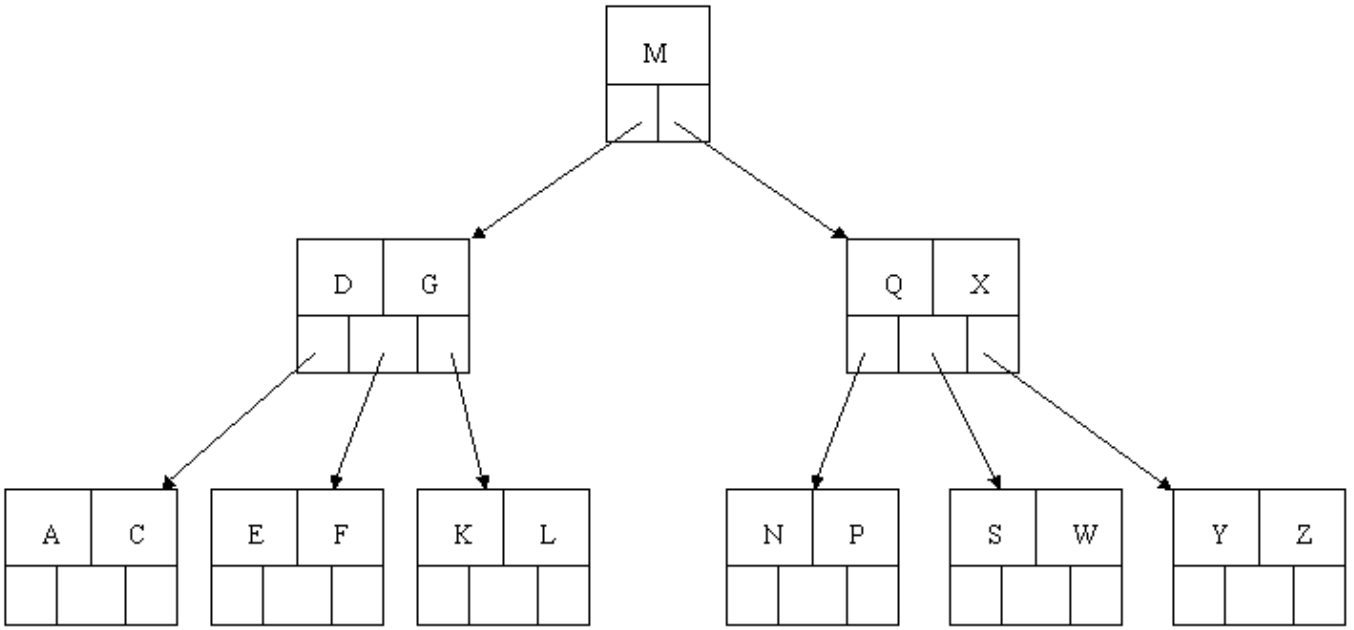
Deletion From Leaf Node: Case 1.b (Situation 3)

- Removal of key value leads to less than $\lceil m/2 \rceil - 1$ keys
  - Neither the left nor the right sibling contains more than $\lceil m/2 \rceil - 1$ keys (situation 3)

- Approach
  - Create a new node with the following keys
    - All the key values from the deficient node
    - All the key values from one of its siblings
    - Key in the parent between the two combined sibling nodes
  - Remove the key from the parent
    - Replace the two children it separated with the combined node
  - If now the number of elements in the parent under the minimum
    - Repeat these steps with parent as deficient node
    - Unless it is the root, since the root is permitted to be deficient

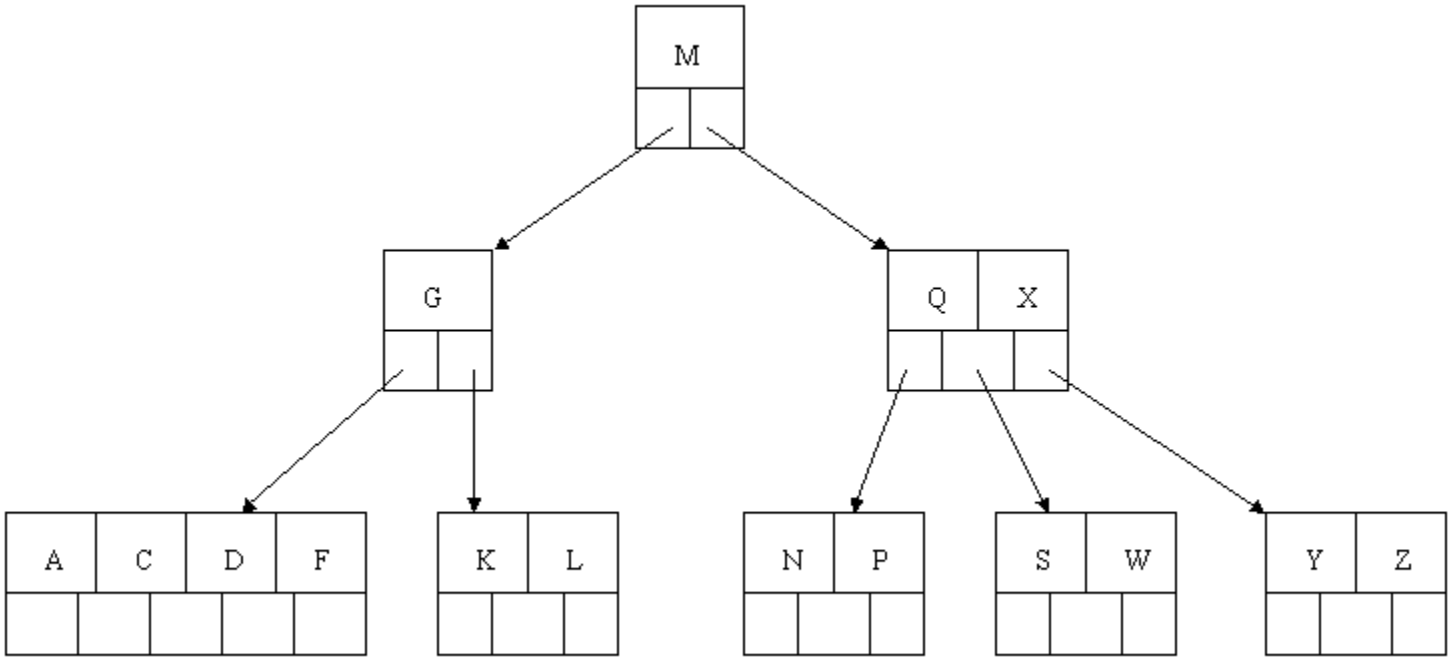
Example: Delete E from B-Tree of order 5

- Let's combine the leaf containing F with the leaf containing A C
  - The key D has to be moved down



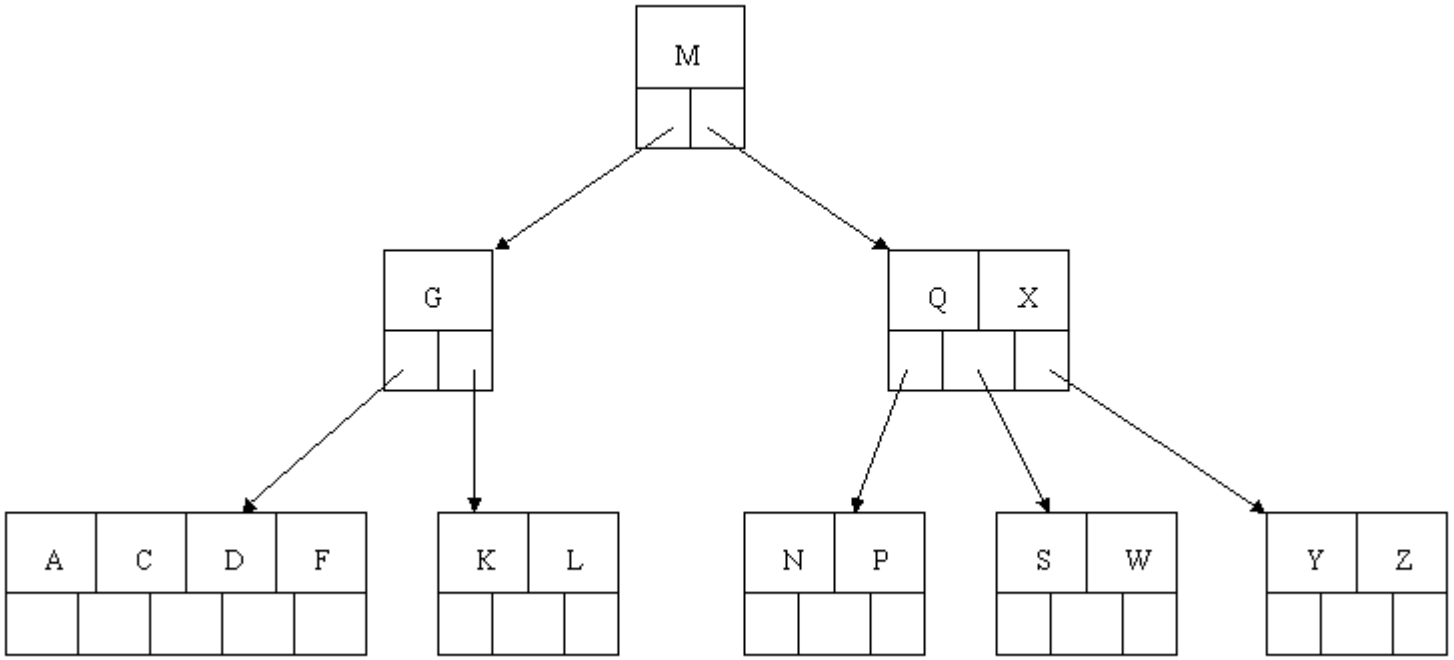
Example: Delete E from B-Tree of order 5

- Let's combine the leaf containing F with the leaf containing A C
  - The key D has to be moved down
- Is this still a B-tree?
  - Parent not contains only one key G, which is not acceptable



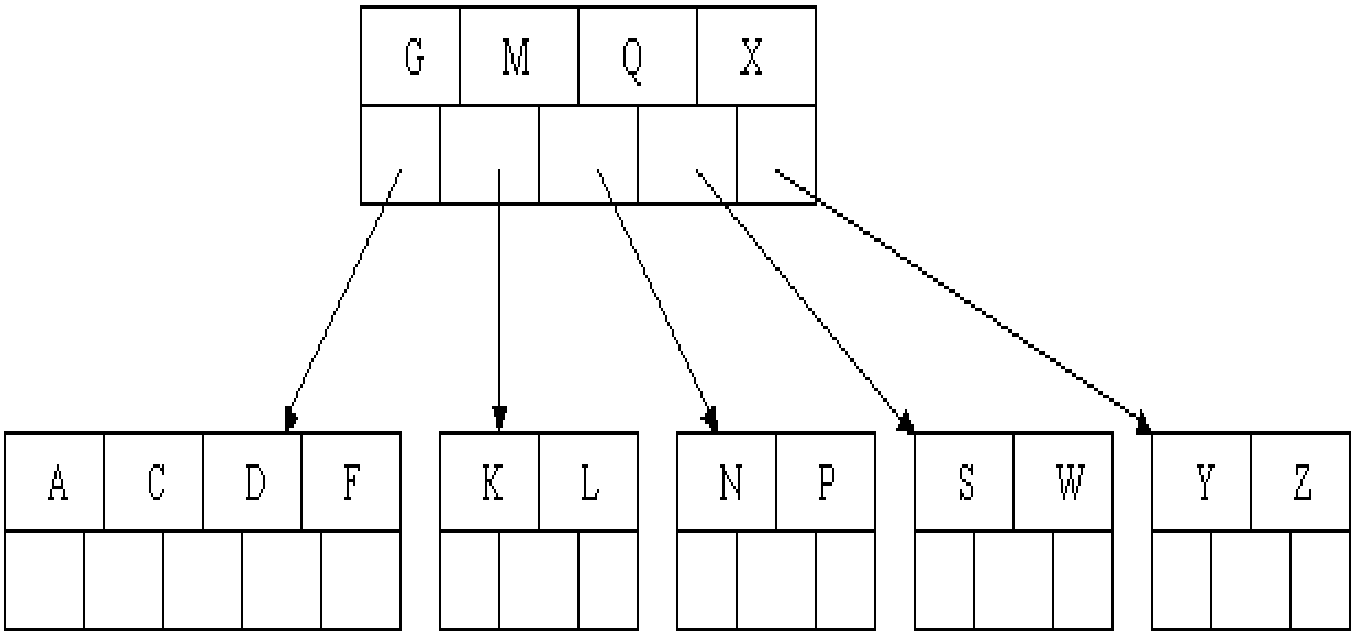
Example: Delete E from B-Tree of order 5

- The siblings do not have more than $\lceil m/2 \rceil - 1$ keys
  - Combine node containing G with one of the siblings
  - Move M down from the parent (root)



Example: Delete E from B-Tree of order 5

- The siblings do not have more than $\lceil m/2 \rceil - 1$ keys
  - Combine node containing G with one of the siblings
  - Move M down from the parent (root)
- The tree shrinks in height by one



Deletion From a Non-Leaf Node: Case 2

- Each key in internal node acts as separation value for two subtrees

- Goal: Find a replacement for the separation key
  - Largest element in the left subtree is still less than the key
  - Smallest element in the right subtree is still greater than the key
  - Either of those elements can be the new key for the two subtrees

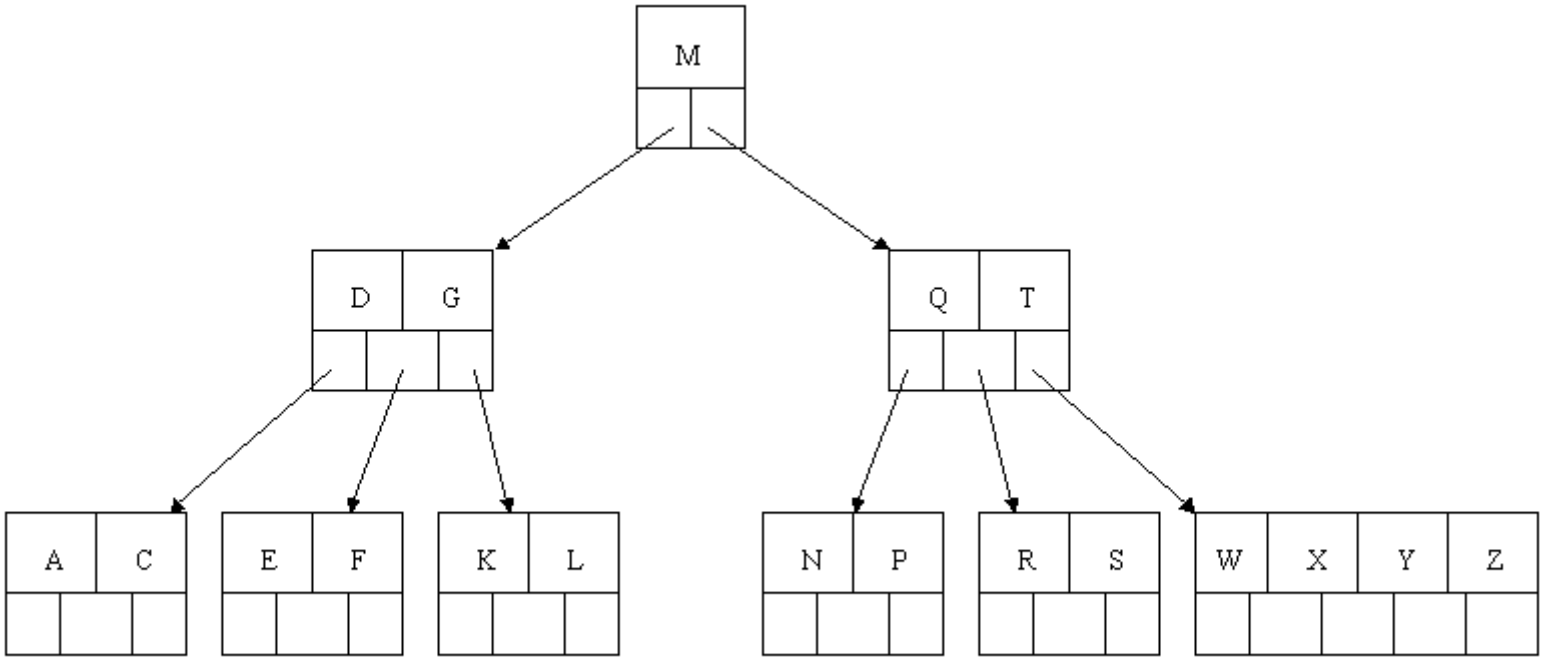
Deletion From a Non-Leaf Node: Case 2

- Choose a new key from one of the subtrees
  - Largest element in the left subtree or
  - Smallest element in the right subtree
- Remove the new key from the node it is in
- Replace the key to be deleted with the new key

- Consequence
  - If the new key belongs to a leaf node
    - Now equivalent to case 1 (Deletion from leaf node)
  - If the new key belong to another internal node
    - Now equivalent to case 2 (Deletion from non-leaf node)

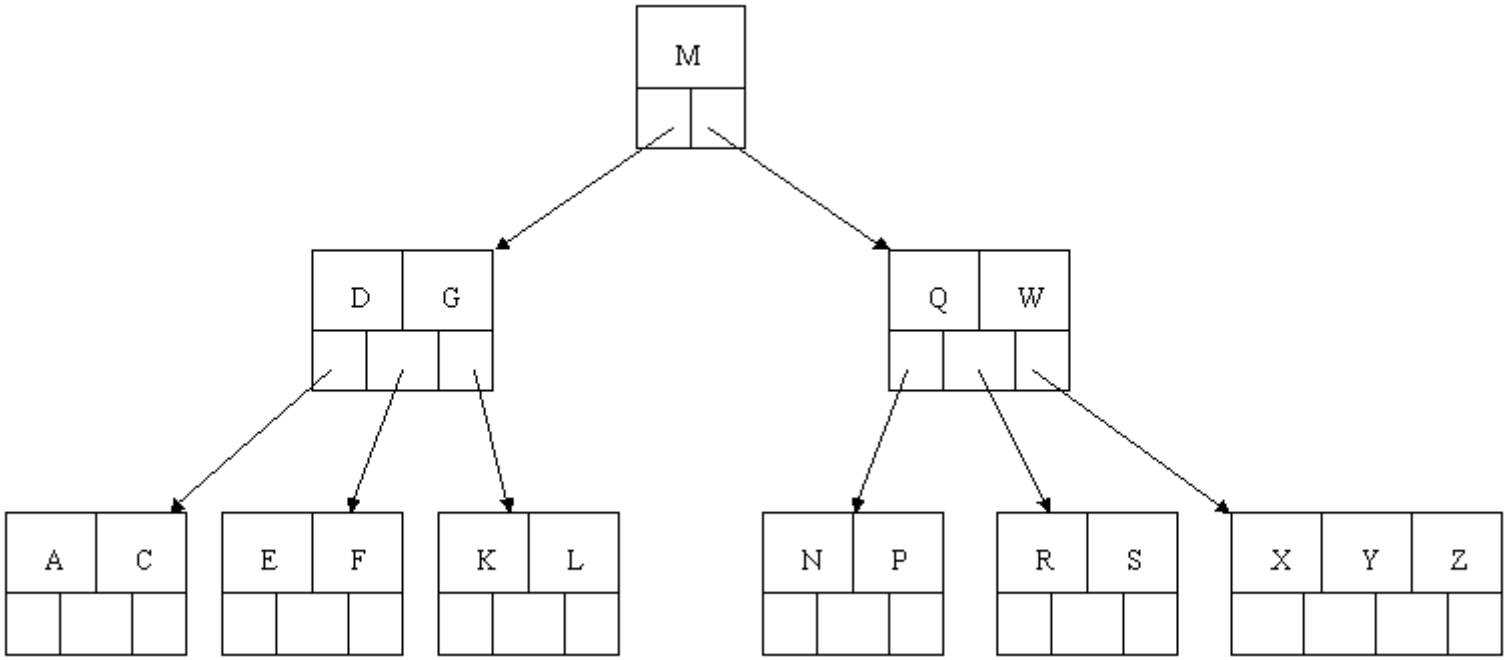
Example: Delete T from B-Tree of order 5

- Since T is not in a leaf, we find its successor
  - Either S or W can replace T



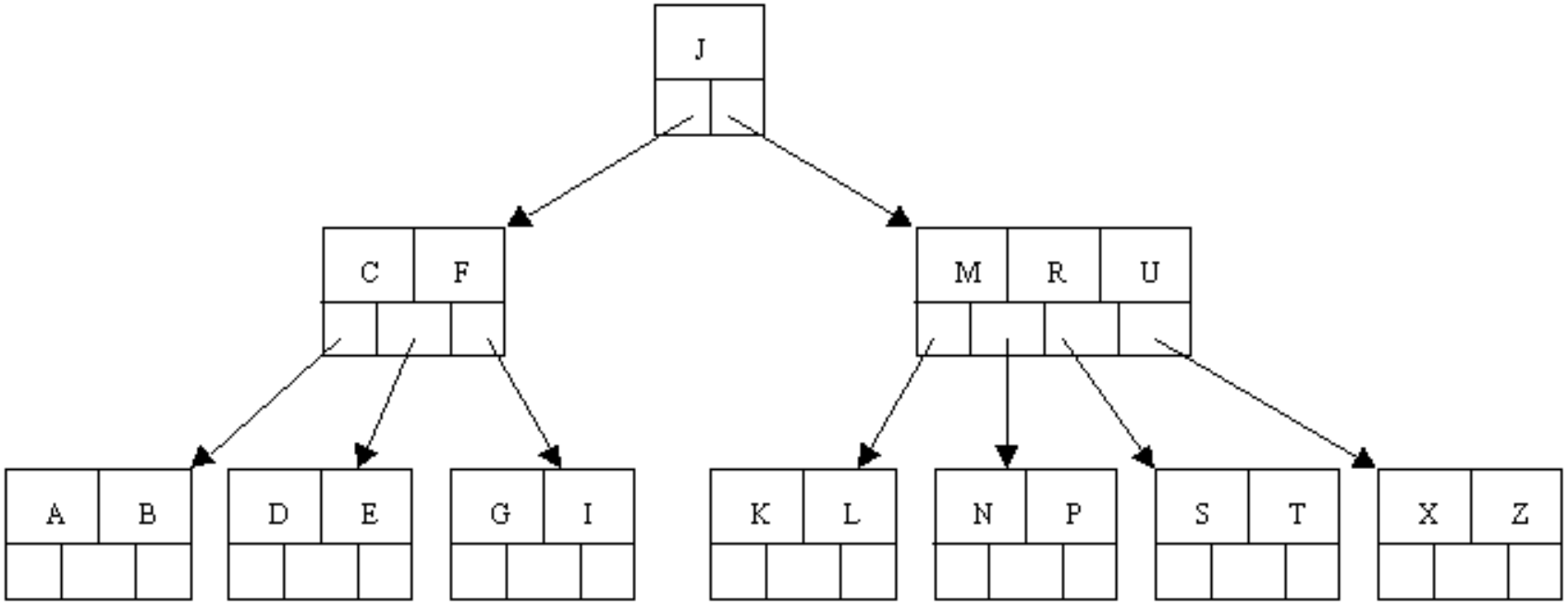
Example: Delete T from B-Tree of order 5

- Select W, and move W up to replace the T
  - Since this leaf has extra keys so no extra operation is required



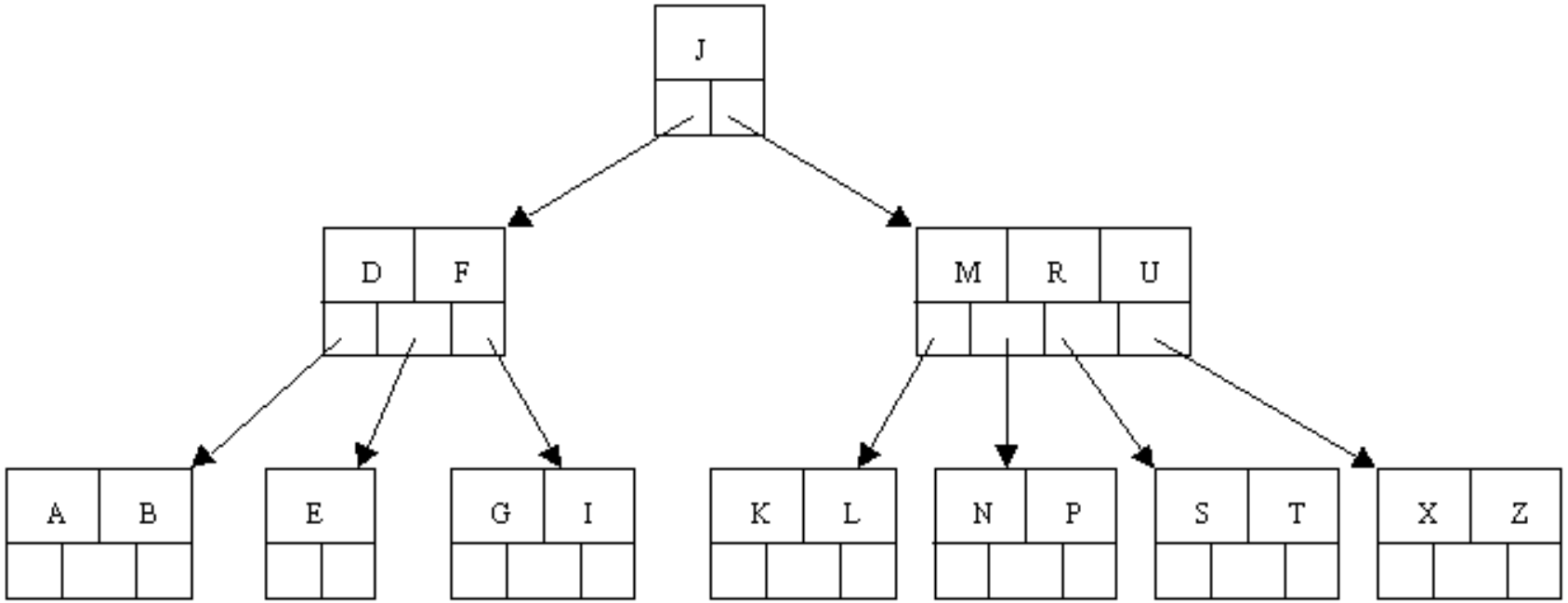
Example: Delete C from B-Tree of order 5

- Move D up to replace C
  - B can equally be used as replacement



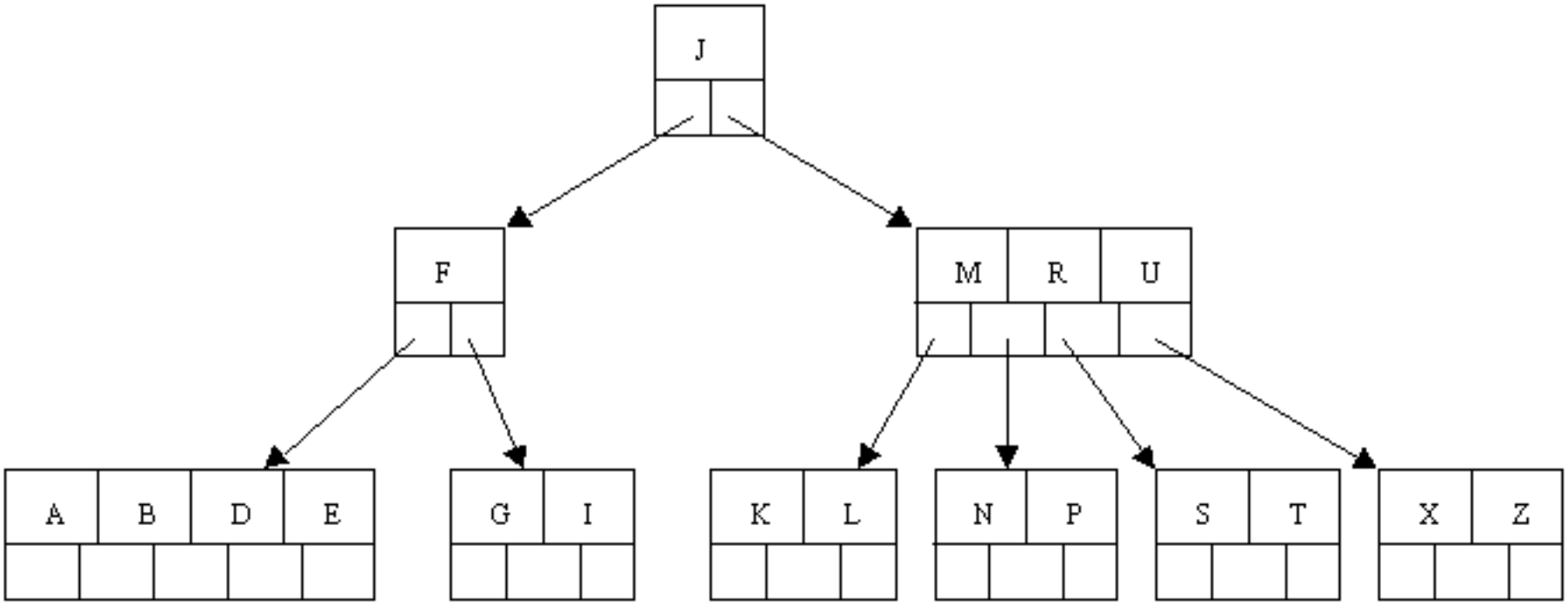
Example: Delete C from B-Tree of order 5

- Moving D will leave a node with less than $\lceil m/2 \rceil - 1 = 2$ keys
  - Equivalent to Case 1



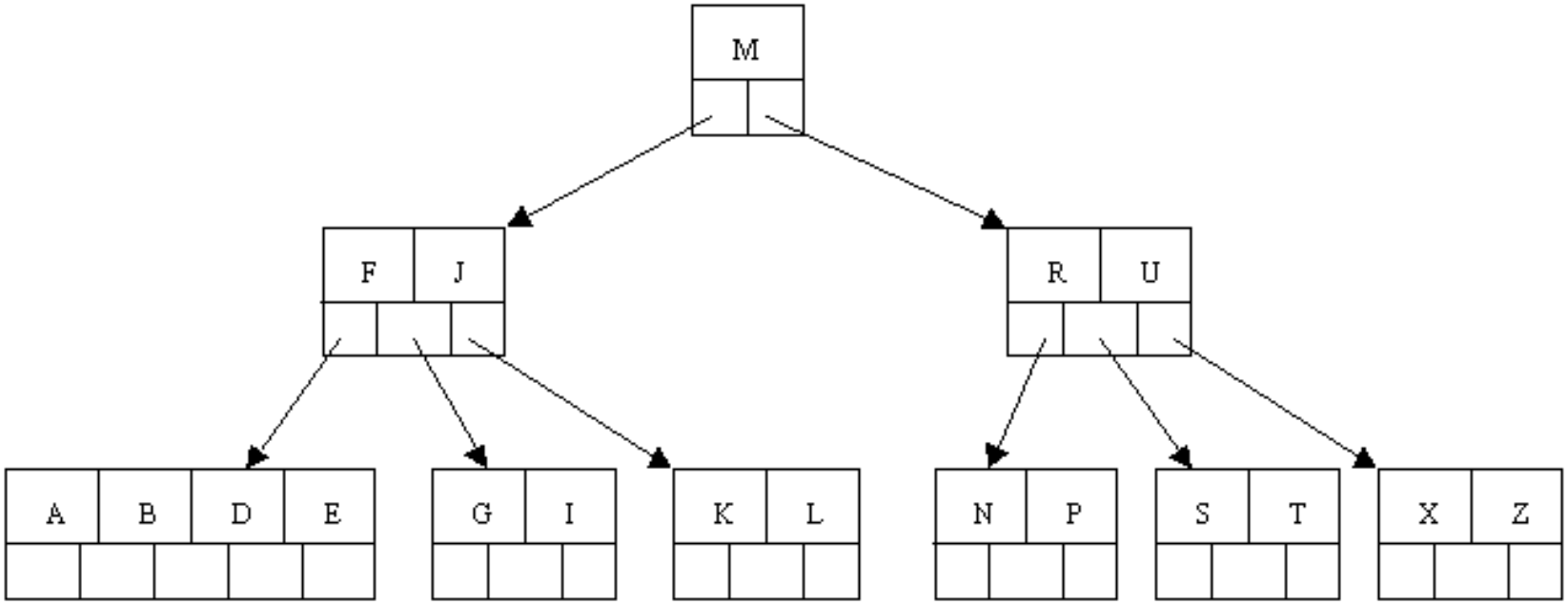
Example: Delete C from B-Tree of order 5

- Neither the sibling to the left or right of the node containing E has an extra key
  - We must combine the node with one of these two siblings



Example: Delete C from B-Tree of order 5

- Now the node containing F does not have enough keys
  - Its right sibling has an extra key
    - Borrow M from the sibling, move it up to the parent
    - Bring the J down to join the F
    - Reattach K L node to the right of the J



Another Example

- B-Tree of order 3
  - 21 7 91 1 38 0 8 11 67 112 2 3 88 4 70

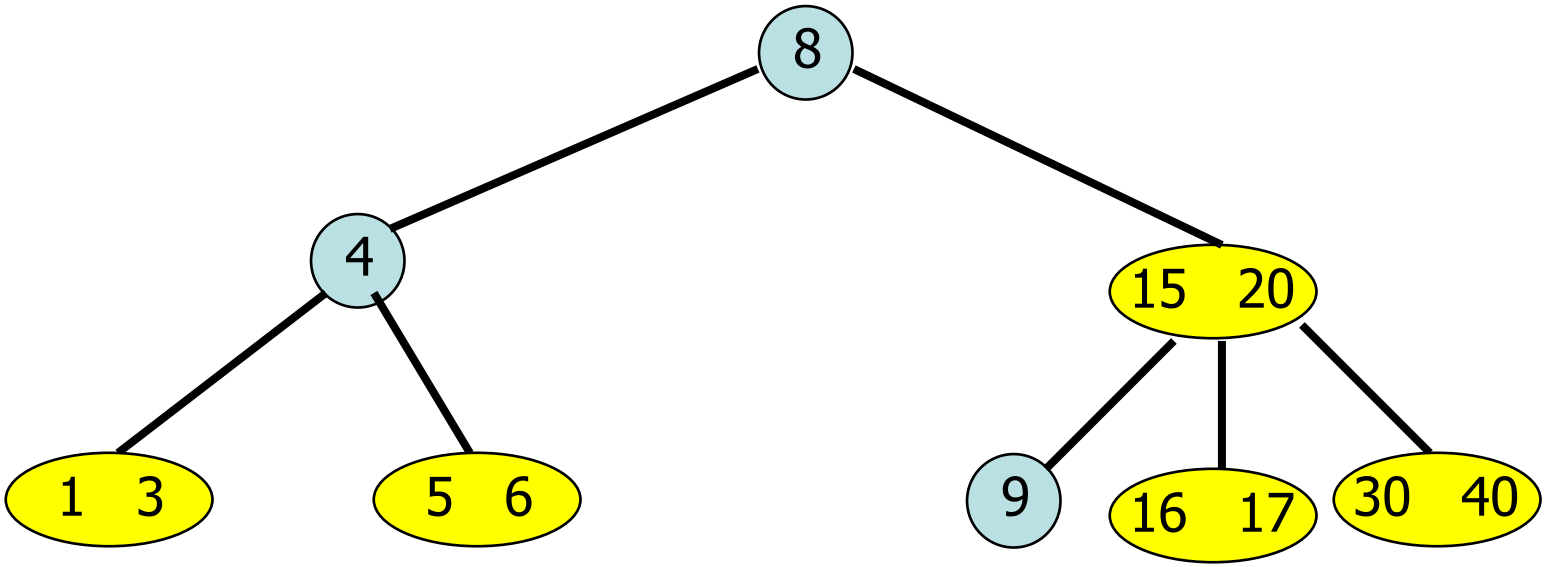
- For same series draw the B-tree with degree 2

- Delete the numbers from the B-tree in the same order as they were inserted

Any Question So Far?



Example: Insertion in B-Tree (Order 3)



Example: AVL Trees

- Number of elements is $n = 2^{30}$ ($\sim 10^9$)
  - Minimum height = 30
  - Maximum height = 43
- Just the pointers requires almost 8 GiB
  - Each node maintains two pointers of 4 bytes each
  - AVL tree has to reside on disk
- When the AVL tree resides on a disk, up to 43 disk access are made for a search
- With 10 ms seek time, this would require approximately 0.43 s
  - Not acceptable